

AI+HPC
利用 NVIDIA Modulus 實踐 PINN 於物理模擬

Jay Chen | Data Scientist @ NVIDIA

Dr. CK Lee | Lead of Taiwan NVAITC @ NVIDIA

AGENDA

09:00-10:15: Review DL Fundamentals

10:30-12:00: Data Driven Lab: Solving Steady State Flow using Neural Networks

12:00-13:00: Launch Break

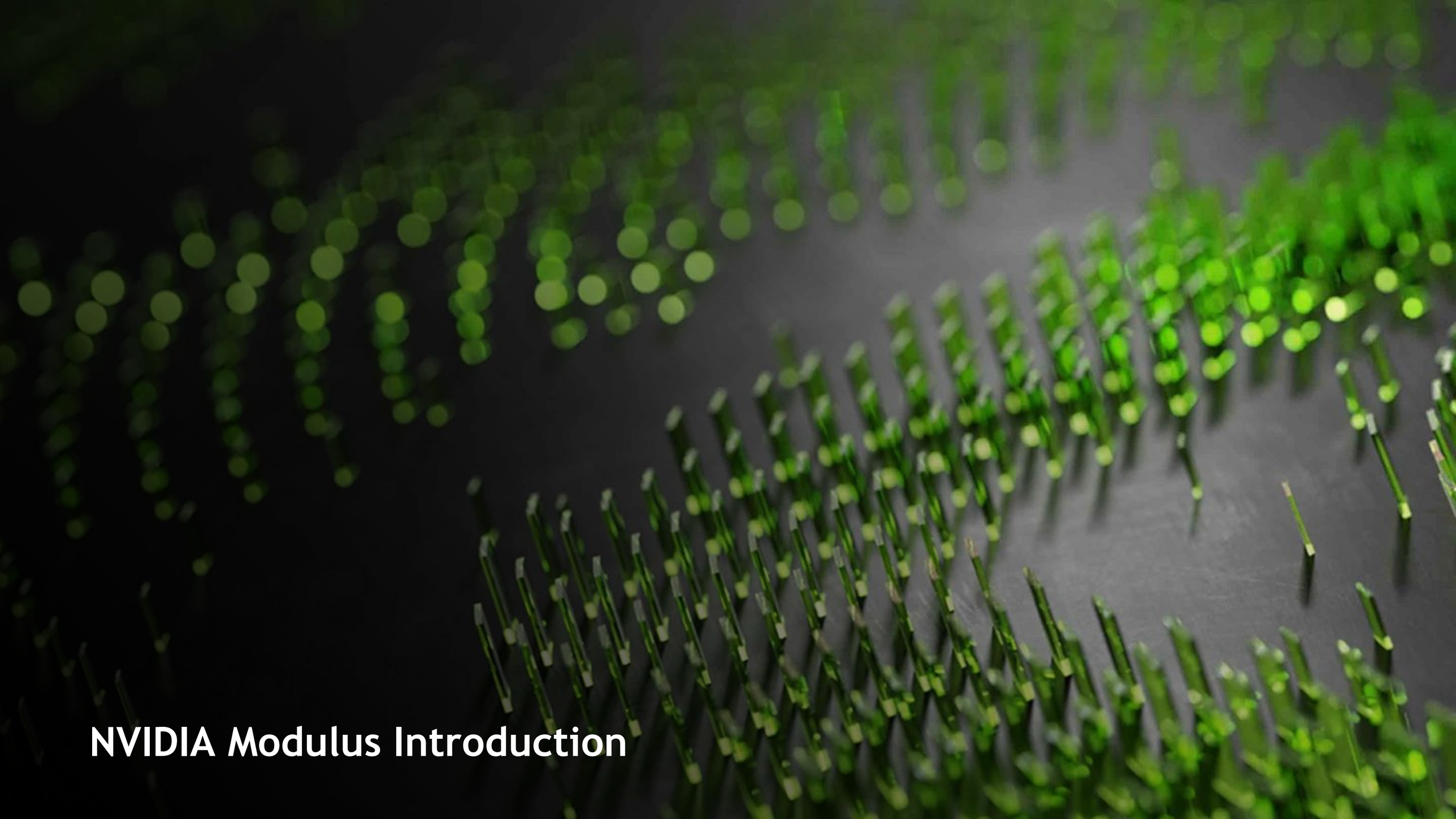
13:00-13:30: Introduce Modulus and PINN

13:45-14:45: Physics Driven Lab 1: Solving Partial Differential Equations

15:00-16:00: Physics Driven Lab 2: Solving transient problems and inverse problems

15:00-16:00: Physics Driven Challenge: solve a fluid mechanics problem




NVIDIA Modulus Introduction

NVIDIA MODULUS

V21.12

https://developer.nvidia.com/modulus

A Framework for Developing Physics Machine Learning Neural Network Models

If you're an engineer, scientist, researcher, or student, using surrogate models is pervasive in your life's work, from basic analysis workflows to building the most complex digital twin applications.

NVIDIA Modulus is a neural network framework that blends the power of physics and partial differential equations (PDEs) with AI to build more robust models for better analysis. Whether you're looking to get started with AI-driven physics problems or designing digital twin models for complex non-linear, multi-physics systems, NVIDIA Modulus can support your work.

Download Now

[Demo]: Accelerating Scientific & Engineering Simulation Workflows with AI



[Demo] Flow Physics Quantification in an Aneurysm Using NVIDIA SimNet



NVIDIA Developer Program Membership Required

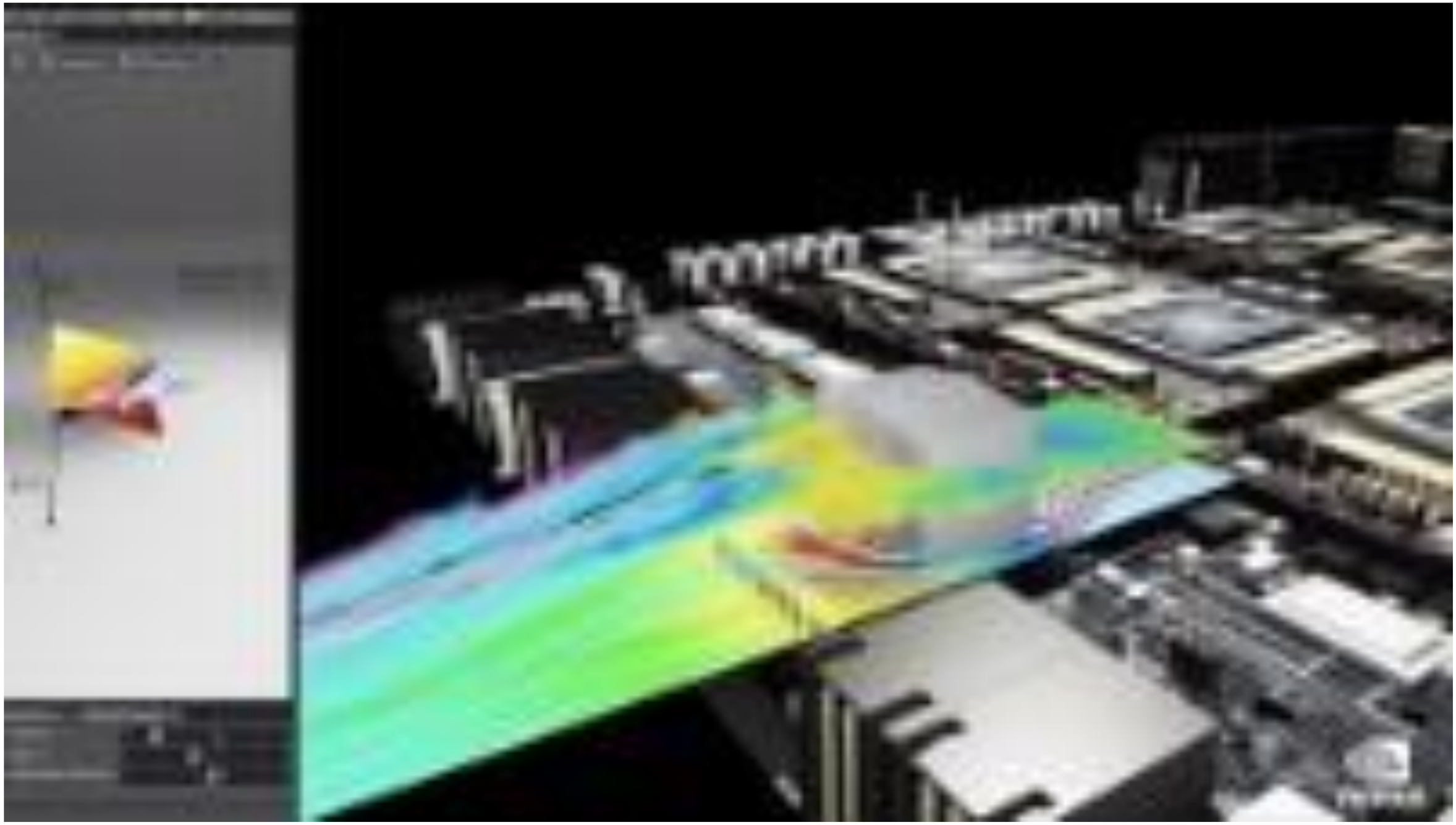
The file or page you have requested requires membership in the NVIDIA Developer Program. Please either log in or join the program to access this material. Learn more about the benefits of the NVIDIA Developer Program.

Login Join now

Docker Container

- Download Modulus 21.06 Container for Linux
- Download Modulus 21.06 Installation Guide
- Download Modulus 21.06 User's Guide
- Download Modulus 21.06 Examples
- Download Modulus 21.06 Release Notes
- Download Modulus 21.06 Source Code Documentation

Compressed Tar Archive



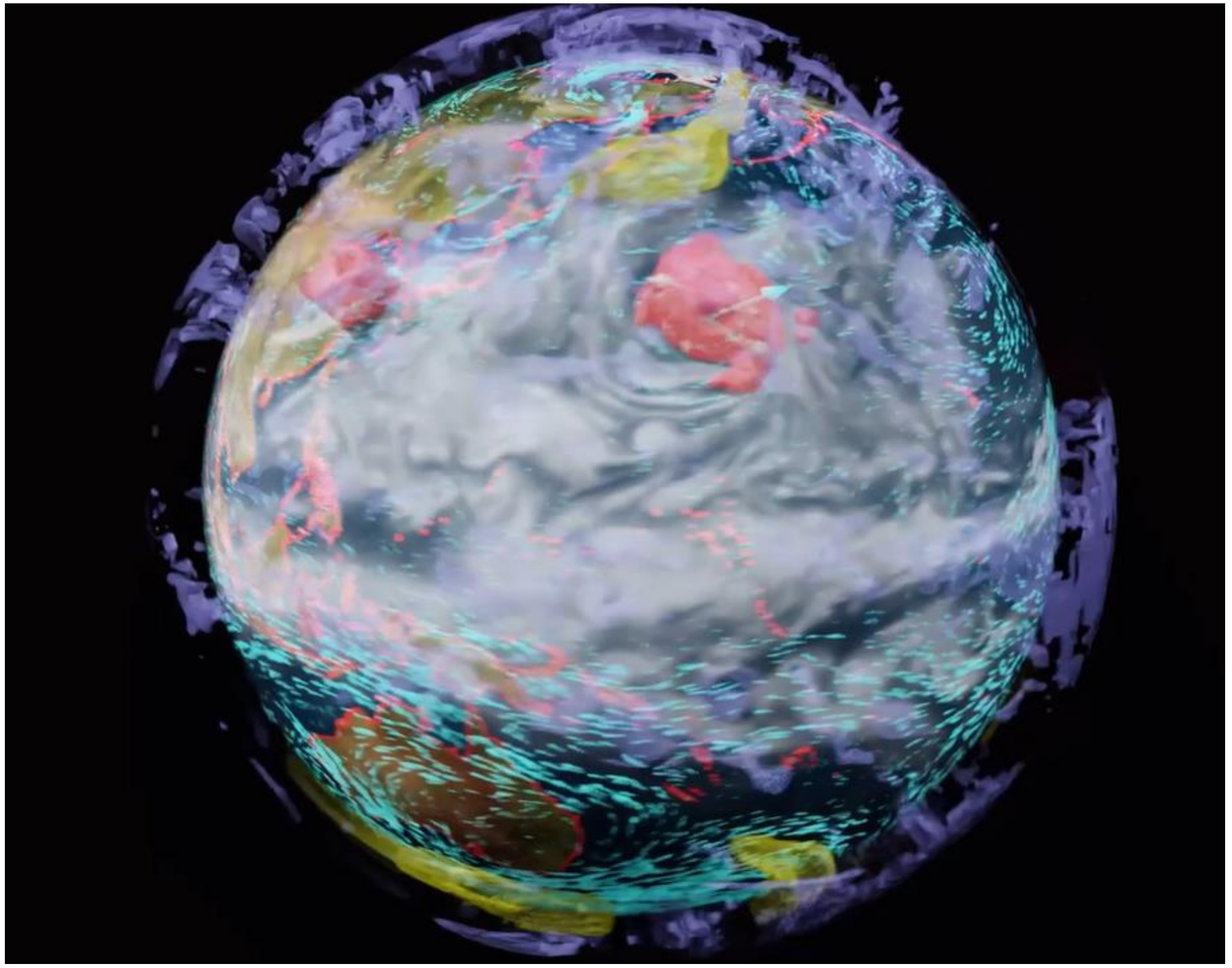
https://www.nvidia.com/en-us/on-demand/session/gtcfall21-d31037/



https://www.nvidia.com/en-us/on-demand/session/gtcfall21-d31038/

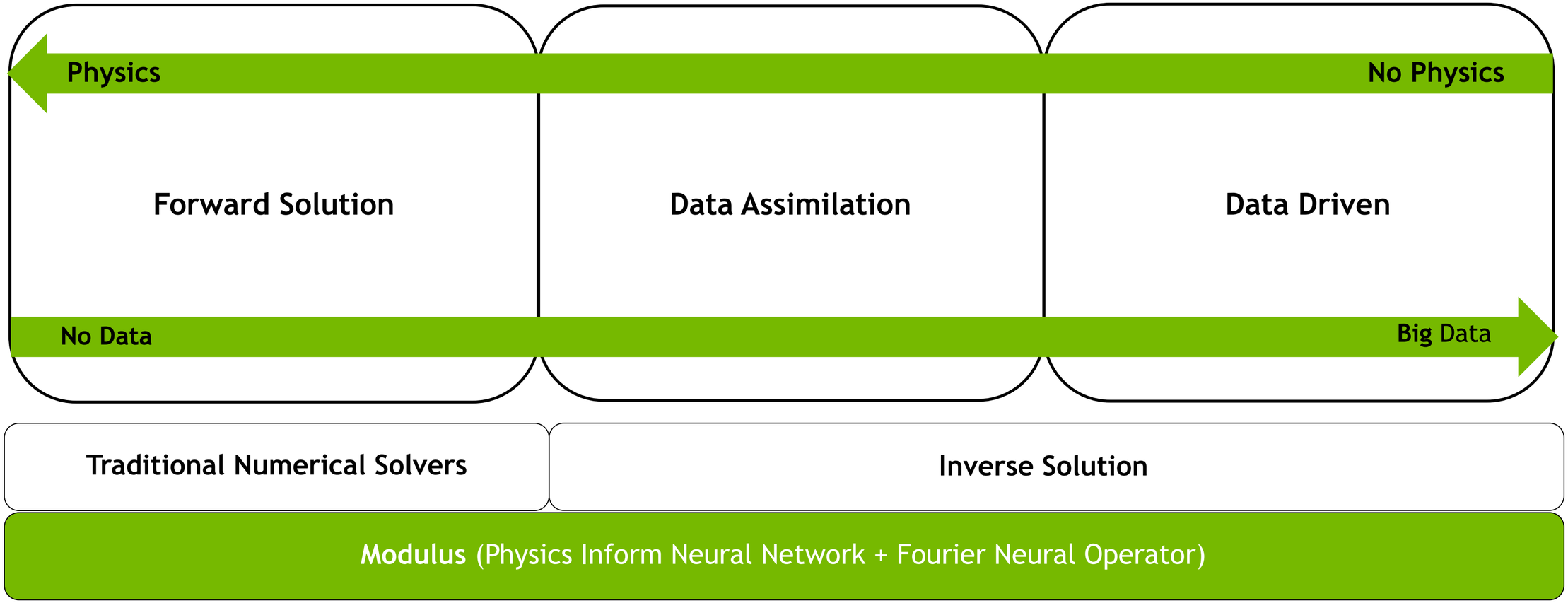
NVIDIA EARTH-2

https://blogs.nvidia.com/blog/2021/11/12/earth-2-supercomputer/



AI IN COMPUTATIONAL SCIENCES

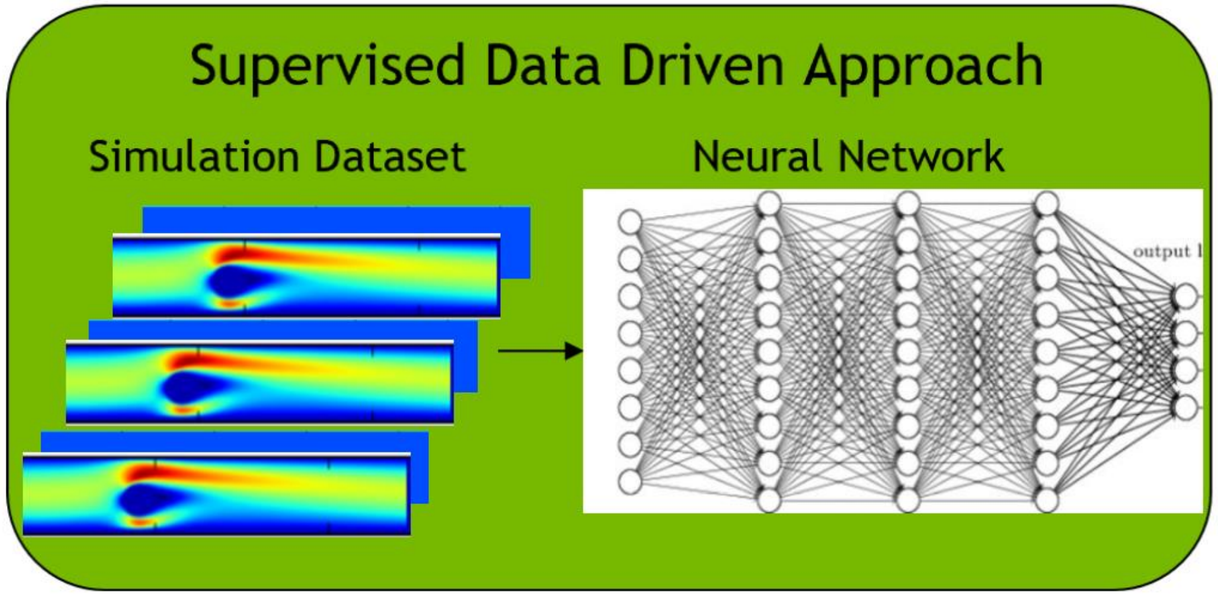
Primary Driver: Data vs. Physics

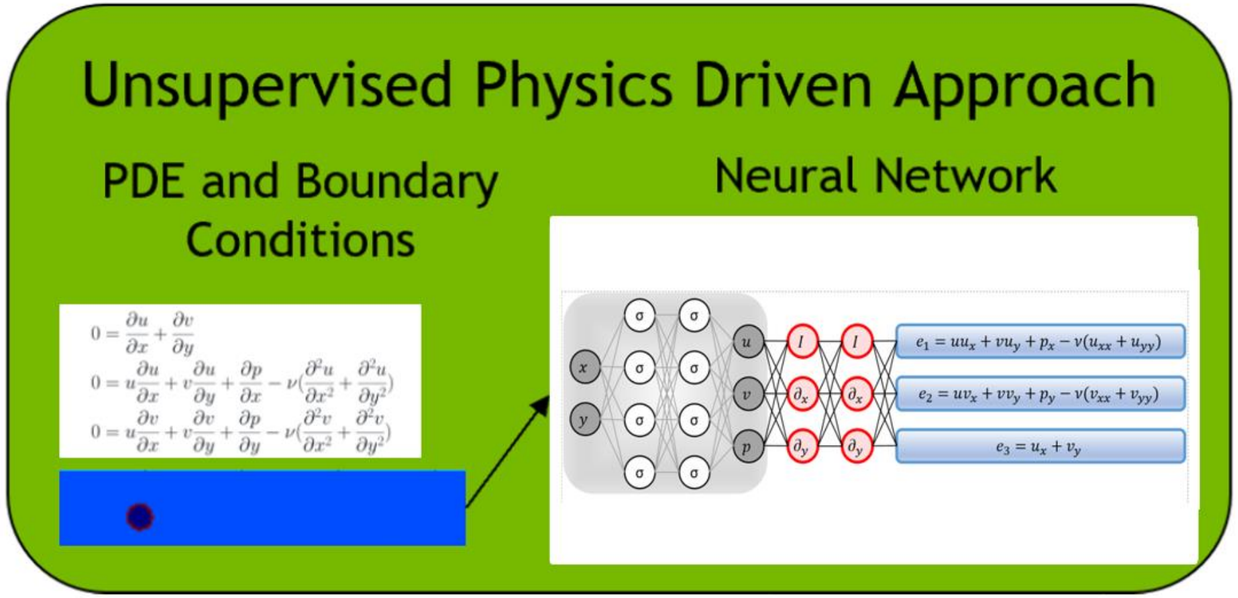


SOLVING PDES WITH NEURAL NETWORKS

A **Data Driven Neural Network** requires training data

A **Physics Driven Neural Network** solver does **NOT** require training data



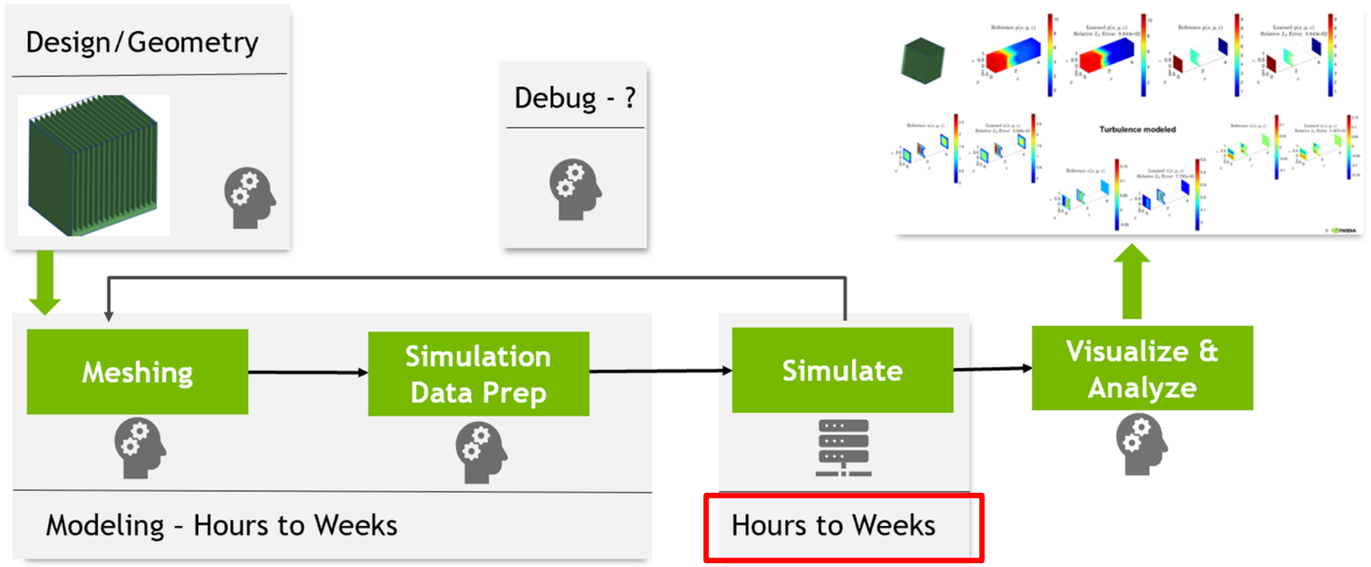


N layers

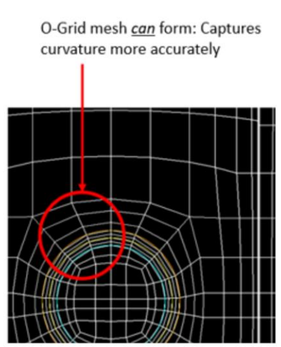
m*N layers (for mth order PDE)

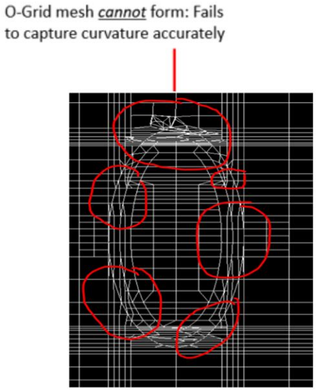
NVIDIA.

ISSUES IN TRADITIONAL SIMULATIONS



- **Meshing Accuracy**

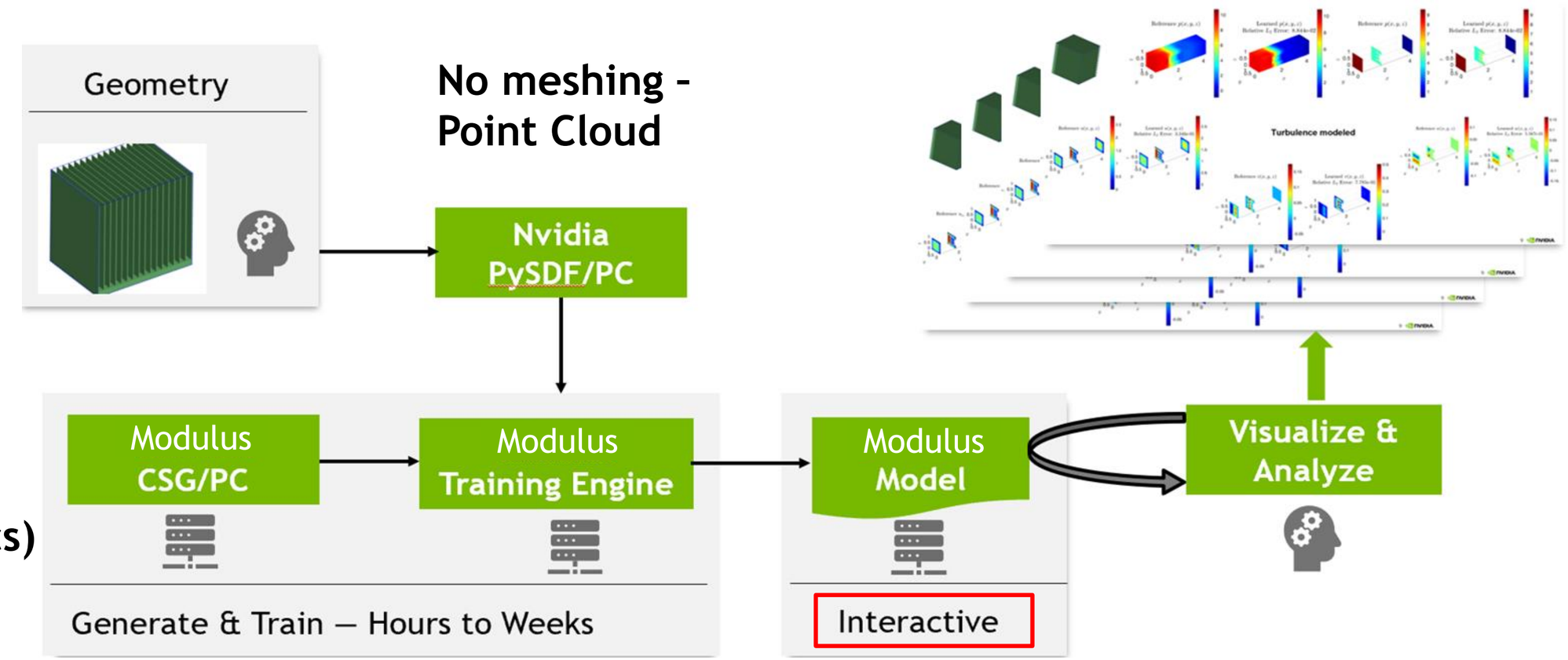




- **Limited Scope & Applicability**
- **Computational Speed Bottleneck**

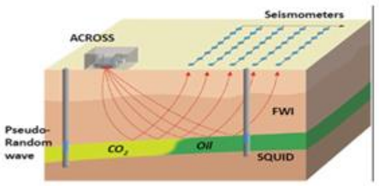
MODULUS – PHYSICS INFORMED NEURAL NETWORK

Parameterization (Geometry/Physics)



AI ENABLING NEXT GENERATION SIMULATION

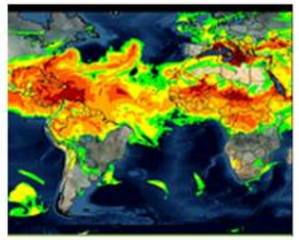
Inverse & Data Assimilation Problems



Oil & Gas

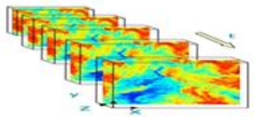


Medical Imaging

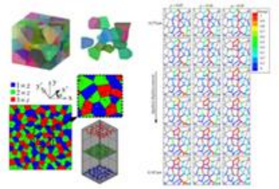


Climate

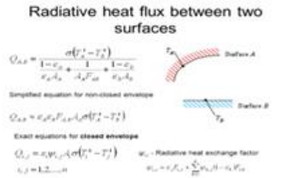
Improved Physics & Predictions



Turbulence



Micro-mechanical Material Model



Radiation

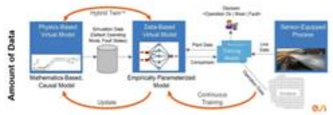
Real Time Simulations



Robotics

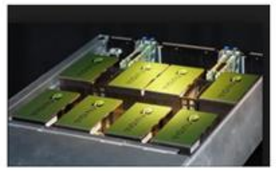


Autonomous Ride & Handling

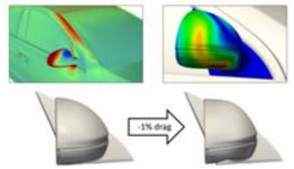


Digital Twin

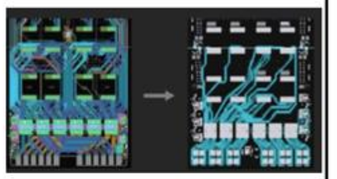
Digital Design & Manufacturing



Heat Sink



Aerodynamics



Vias on a PCB

Physics & Data - No Traditional Solver

Physics - Traditional Solver (Speed is a limitation)

AGENDA

09:00-10:15: Review DL Fundamentals

10:30-12:00: Data Driven Lab: Solving Steady State Flow using Neural Networks

12:00-13:00: Launch Break

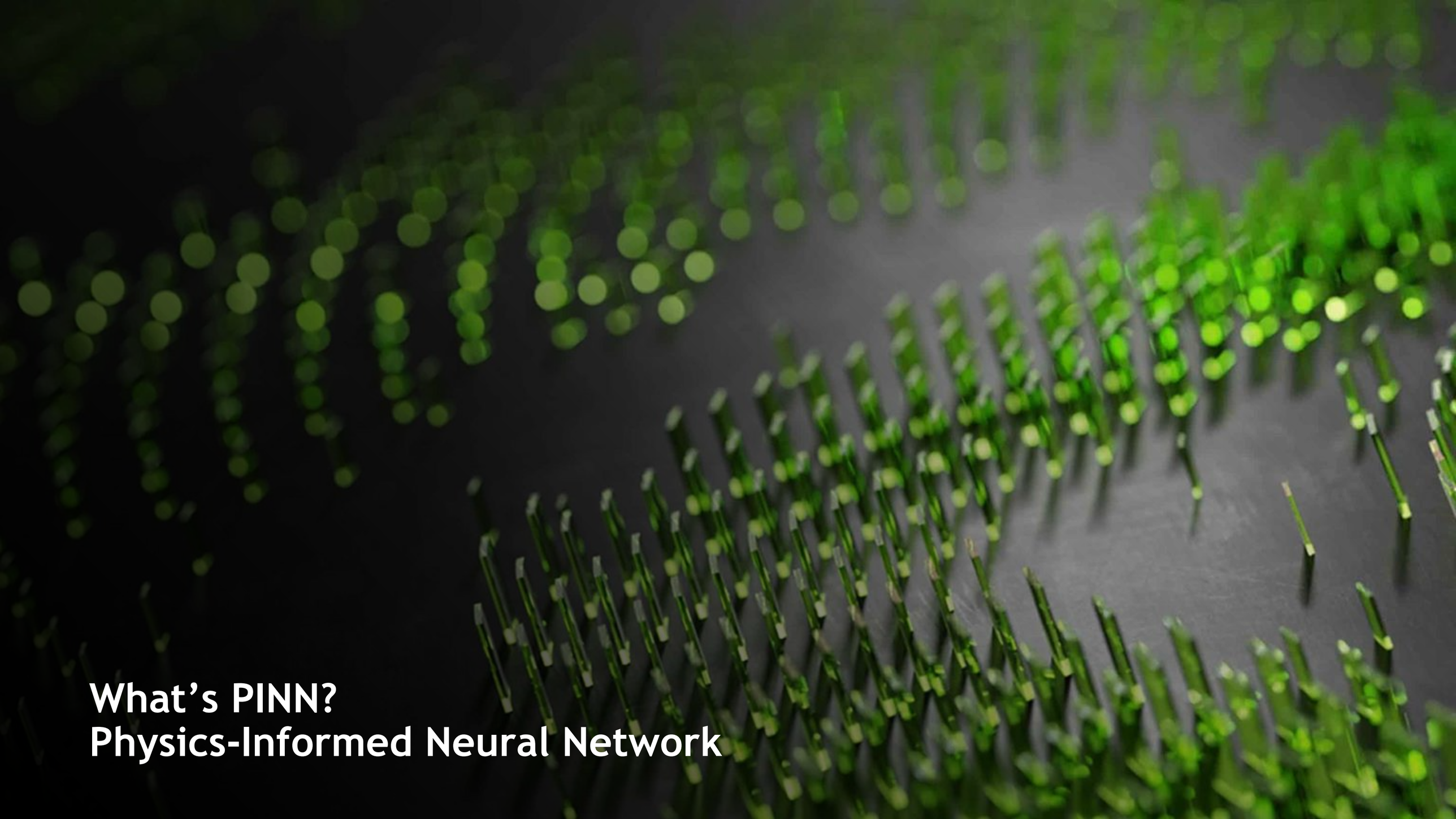
13:00-13:30: Introduce Modulus and PINN

13:45-14:45: Physics Driven Lab 1: Solving Partial Differential Equations

15:00-16:00: Physics Driven Lab 2: Solving transient problems and inverse problems

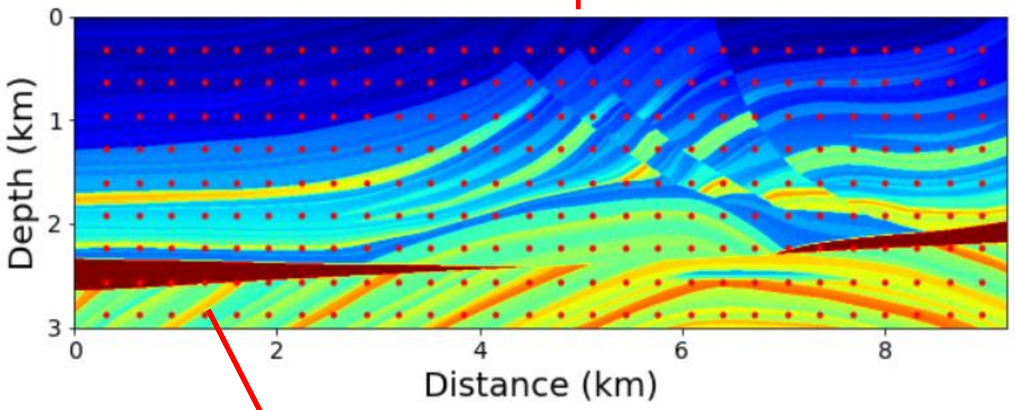
15:00-16:00: Physics Driven Challenge: solve a fluid mechanics problem




What’s PINN?
Physics-Informed Neural Network

AN EXAMPLE: SEISMIC PROBLEMS

Point cloud representation



Hard constraint to IC
Multiply by t^2 to u to enforce $u(x, z, t = 0) = 0$

PHYSICS

Neural Network

Input

Output

PDE

$\frac{\partial^2 u}{\partial t^2} - c^2\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial z^2}\right)$

IC

$u(x, z, t) - u_D$

BC

$\frac{\partial u(x,z,t)}{\partial n} - u_N$

$L_{PDE} + L_{IC} + L_{BC}$

Loss

NEURAL NETWORK SOLVER THEORY

Goal: Train a neural network to satisfy the boundary conditions and differential equations by constructing an appropriate loss function

- Consider an example problem:

$$\mathbf{P}: \begin{cases} \dfrac{\delta^2 u}{\delta x^2}(x) = f(x) \\ u(0) = u(1) = 0 \end{cases}$$

- We construct a neural network $u_{net}(x)$ which has a single value input $x \in \mathbb{R}$ and single value output $u_{net}(x) \in \mathbb{R}$.
- We assume the neural network is infinitely differentiable $u_{net} \in C^{\infty}$ - Use activation functions that are infinitely differentiable

This material is released by NVIDIA Corporation under the Creative Commons Attribution 4.0 International (CC BY 4.0)

NEURAL NETWORK SOLVER THEORY

- Construct the loss function. We can compute the second order derivatives $\left(\frac{\delta^2 u_{net}}{\delta x^2}(x)\right)$ using Automatic differentiation

$$L_{BC} = u_{net}(0)^2 + u_{net}(1)^2$$

$$L_{Residual} = \int_0^1 \left(\frac{\delta^2 u_{net}}{\delta x^2}(x) - f(x)\right)^2 dx \approx \left(\int_0^1 dx\right)\frac{1}{N}\sum_{i=0}^{N}\left(\frac{\delta^2 u_{net}}{\delta x^2}(x_i) - f(x_i)\right)^2$$

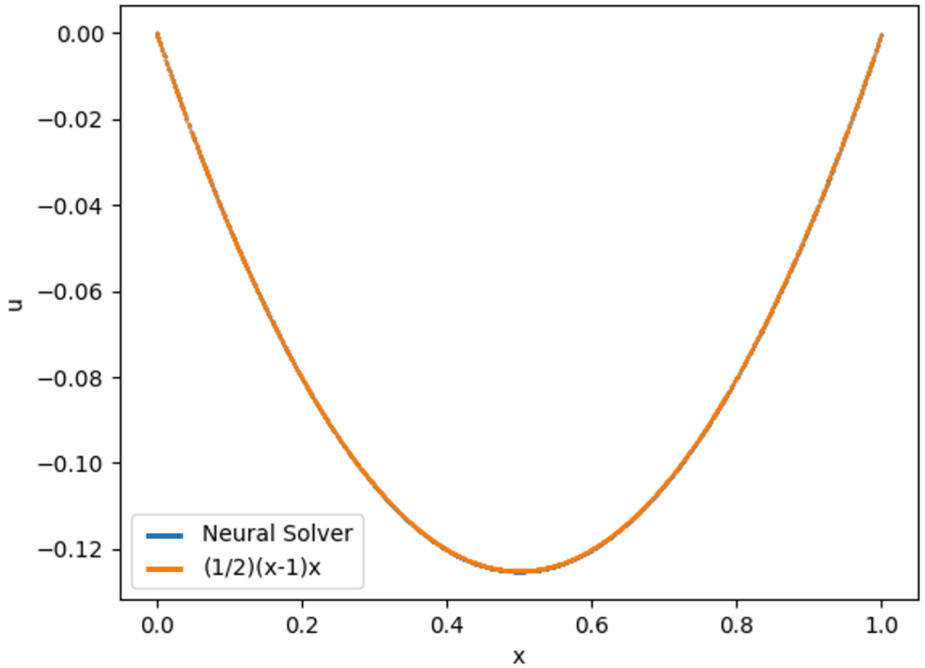
- Where x_i are a batch of points in the interior $x_i \in (0, 1)$. Total loss becomes $L = L_{BC} + L_{Residual}$
- Minimize the loss using optimizers like Adam

This material is released by NVIDIA Corporation under the Creative Commons Attribution 4.0 International (CC BY 4.0)

NVIDIA.

NEURAL NETWORK SOLVER THEORY

- For $f(x) = 1$, the true solution is $\frac{1}{2}(x-1)x$. After sufficient training we have,



Comparison of the solution predicted by Neural Network with the analytical solution

This material is released by NVIDIA Corporation under the Creative Commons Attribution 4.0 International (CC BY 4.0)

NVIDIA.

SOLVING PARAMETERIZED PROBLEMS

- Consider the parameterized version of the same problem as before. Suppose we want to determine how the solution changes as we move the position on the boundary condition $u(l) = 0$
- Parameterize the position by variable $l \in [1, 2]$ and the problem now becomes:

$$\mathbf{P}: \begin{cases} \dfrac{\delta^2 u}{\delta x^2}(x) = f(x) \\ u(0) = u(l) = 0 \end{cases}$$

- This time, we construct a neural network $u_{net}(x, l)$ which has x and l as input and single value output $u_{net}(x, l) \in \mathbb{R}$.
- The losses become

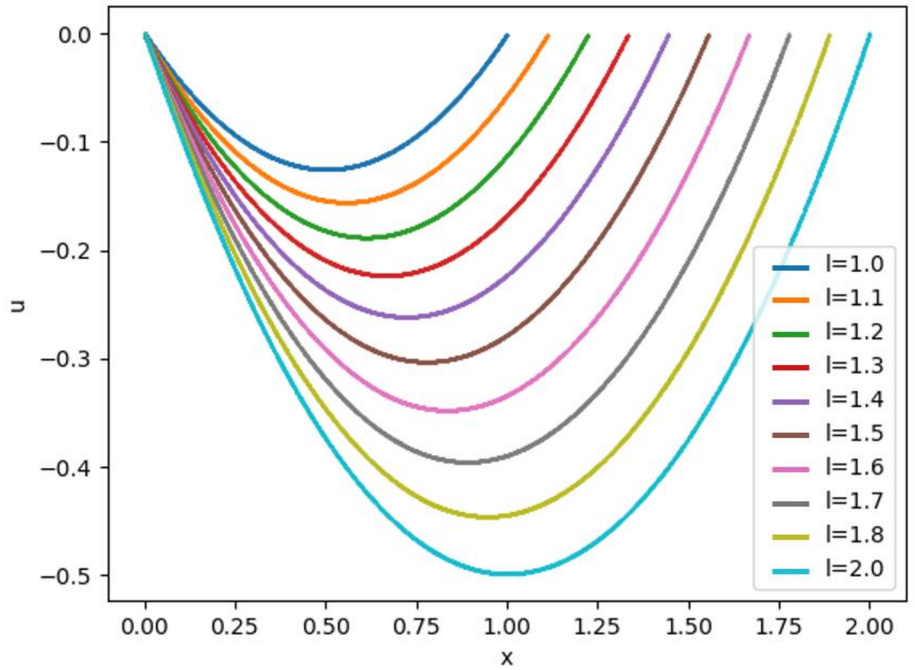
$$L_{Residual} = \int_1^2 \int_0^1 \left(\frac{\delta^2 u_{net}}{\delta x^2}(x) - f(x) \right)^2 dx dl \approx \left(\int_1^2 \int_0^1 dx\, dl \right) \frac{1}{N} \sum_{i=0}^{N} \left(\frac{\delta^2 u_{net}}{\delta x^2}(x_i, l_i) - f(x_i) \right)^2$$

$$L_{BC} = \int_1^2 \big(u_{net}(0, l)\big)^2 + \big(u_{net}(l, l)\big)^2 dl \approx \left(\int_1^2 dl \right) \frac{1}{N} \sum_{i=0}^{N} (u_{net}(0, l_i))^2 + (u_{net}(l_i, l_i))^2$$

This material is released by NVIDIA Corporation under the Creative Commons Attribution 4.0 International (CC BY 4.0)

SOLVING PARAMETERIZED PROBLEMS

- For $f(x) = 1$, for different values of l we have different solutions



Solution to the parametric problem

This material is released by NVIDIA Corporation under the Creative Commons Attribution 4.0 International (CC BY 4.0)

SOLVING INVERSE PROBLEMS

$$\mathbf{P}: \begin{cases} \frac{\delta^2 u}{\delta x^2}(x) = f(x) \\ u(0) = u(1) = 0 \end{cases}$$

- For inverse problems, we start with a set of observations and then calculate the causal factors that produced them
- For example, suppose we are given the solution $u_{true}(x)$ at 100 random points between 0 and 1 and we want to determine the $f(x)$ that is causing it
- Train two networks $u_{net}(x)$ and $f_{net}(x)$ to approximate $u(x)$ and $f(x)$

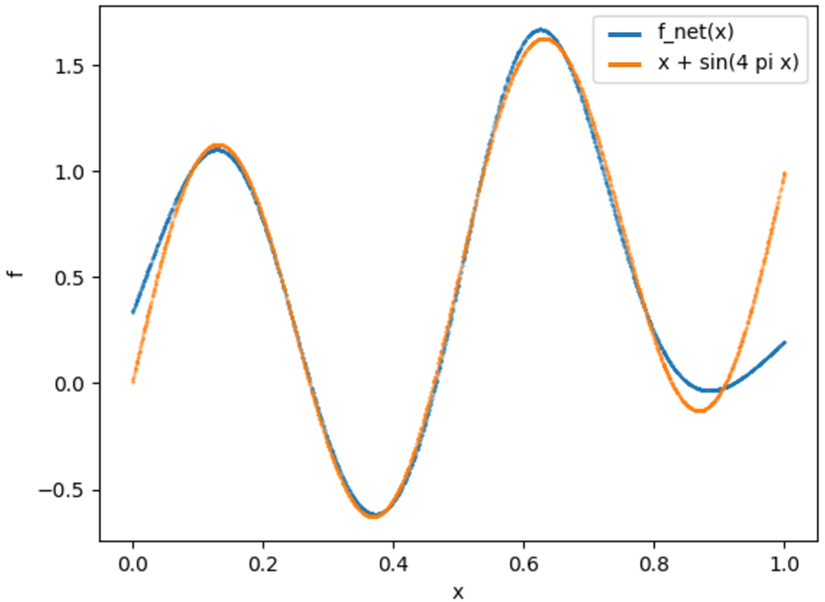
$$L_{Residual} \approx \left(\int_0^1 dx\right) \frac{1}{N} \sum_{i=0}^{N} \left(\frac{\delta^2 u_{net}}{\delta x^2}(x_i) - f_{net}(x_i)\right)^2$$

$$L_{Data} = \frac{1}{100} \sum_{i=0}^{100} \left(u_{net}(x_i) - u_{true}(x_i)\right)^2$$

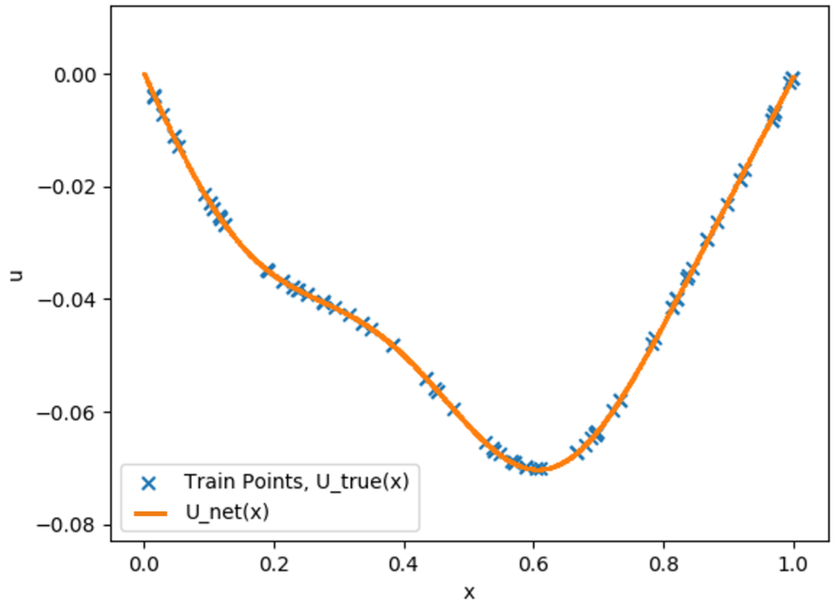
This material is released by NVIDIA Corporation under the Creative Commons Attribution 4.0 International (CC BY 4.0)

SOLVING INVERSE PROBLEMS

- For $u_{true}(x) = \frac{1}{48}\left(8x\left(-1+x^2\right) - \frac{3\sin(4\pi x)}{\pi^2}\right)$ the solution for $f(x)$ is $x + \sin(4\pi x)$



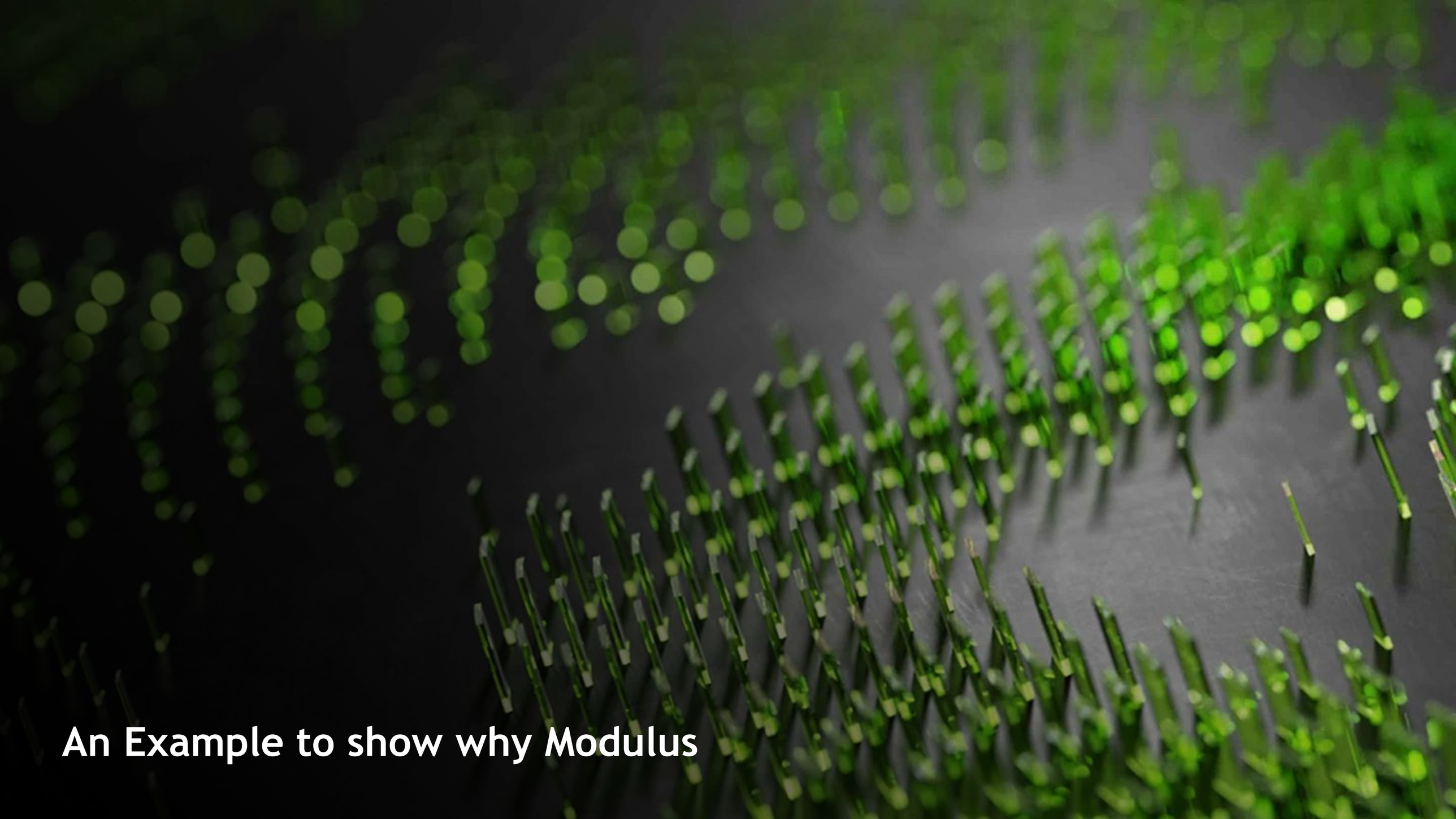
Comparison of the true solution for $f(x)$ and the $f_{net}(x)$ inverted out



Comparison of $u_{net}(x)$ and train points from u_{true}

This material is released by NVIDIA Corporation under the Creative Commons Attribution 4.0 International (CC BY 4.0)

NVIDIA.

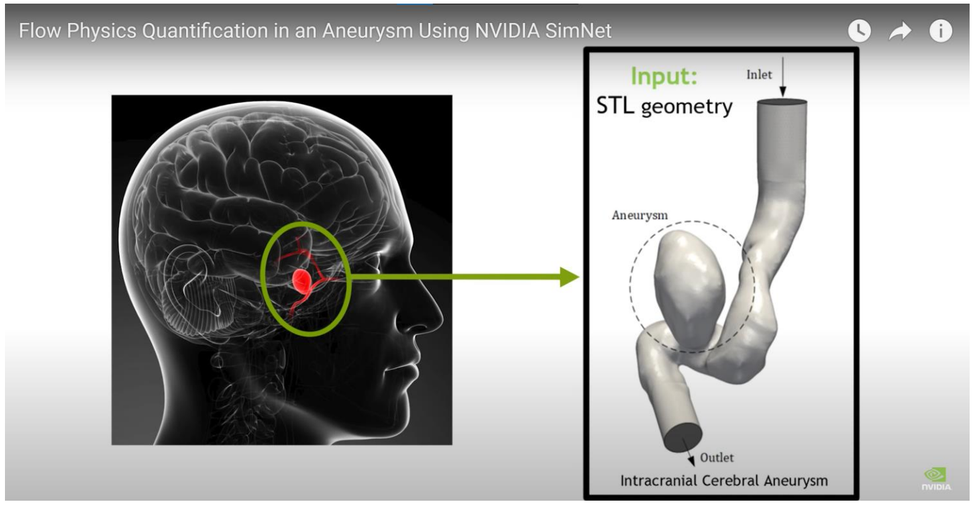


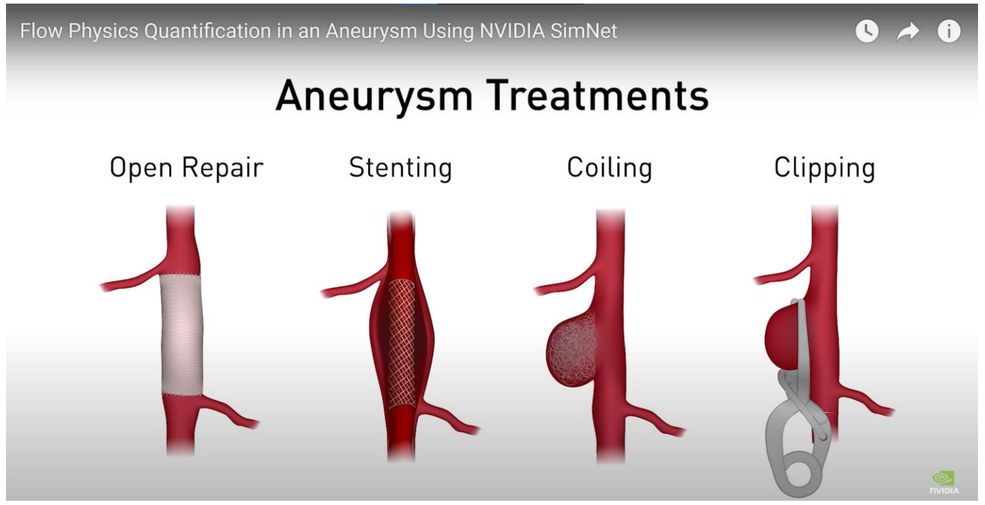
An Example to show why Modulus

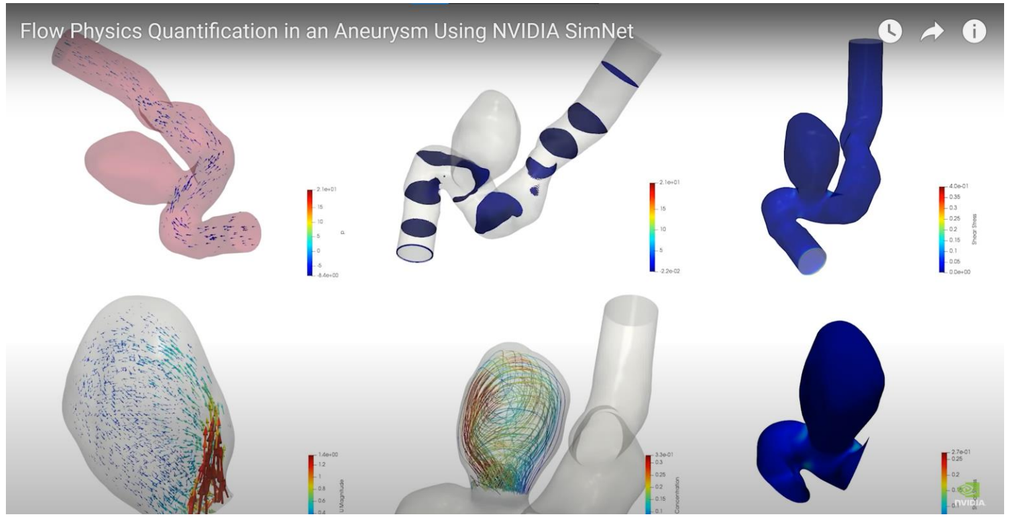
TRANSIENT: MEDICAL IMAGING OF AN ICA

FORWARD SIMULATION

https://www.youtube.com/watch?v=QjY_8xFjsgE

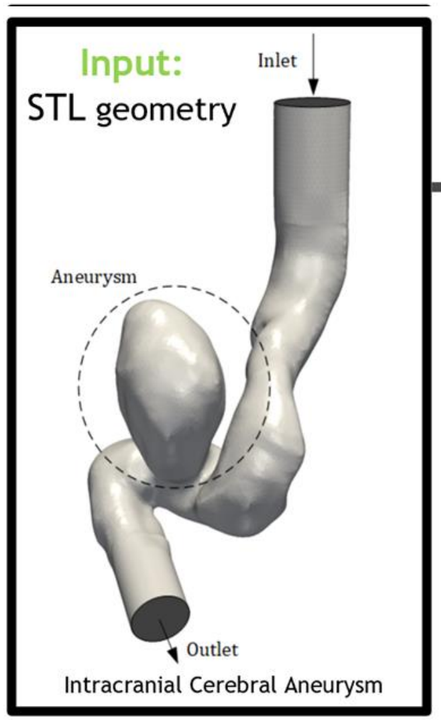


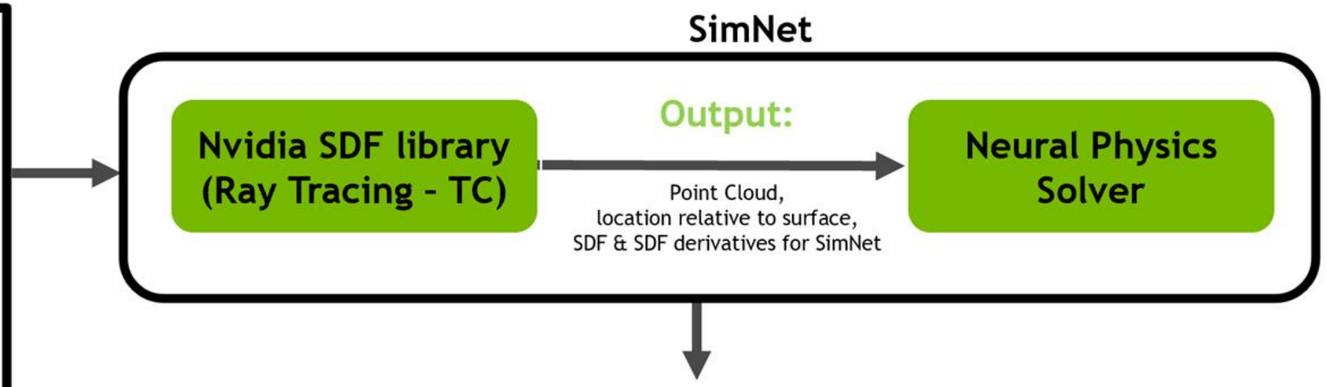


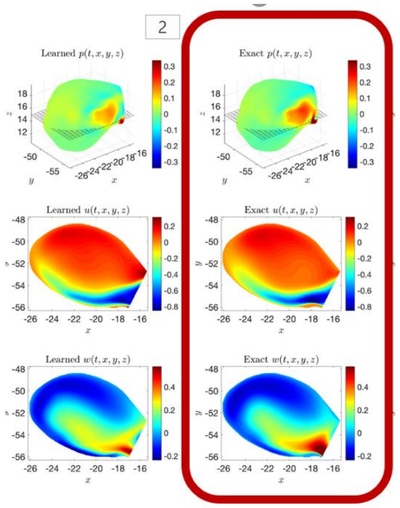


TRANSIENT: MEDICAL IMAGING OF AN ICA

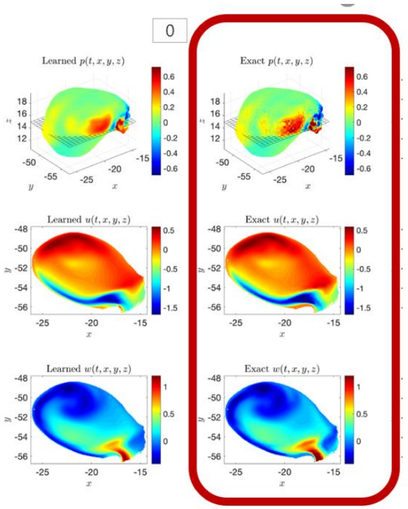
FORWARD SIMULATION







SimNet vs. OpenFOAM



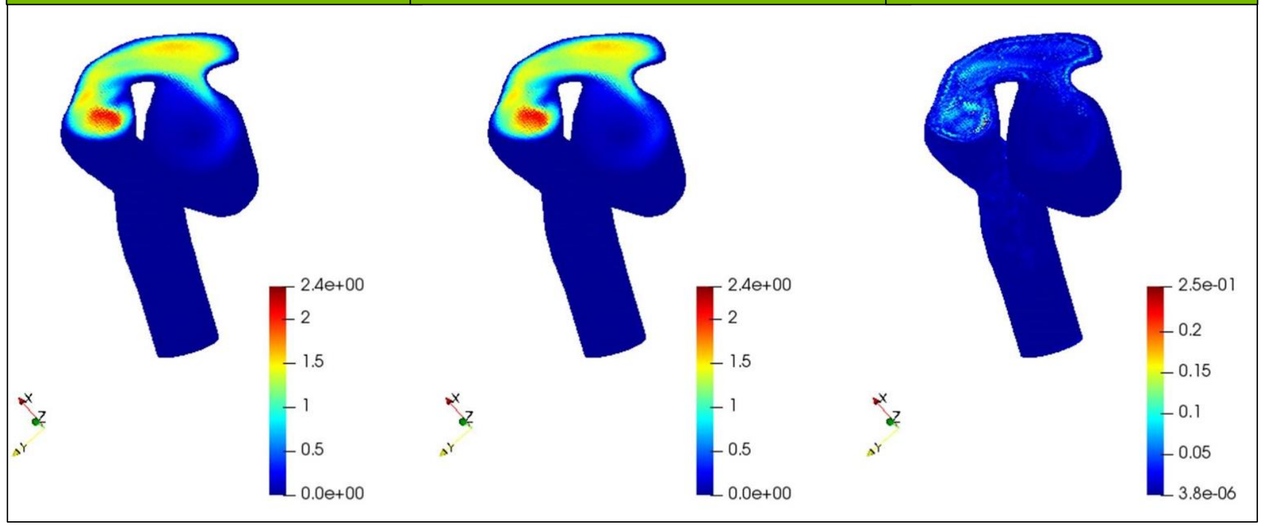
SimNet vs. Nektar++

NVIDIA.

ICA — COMPARISON BETWEEN SIMNET & OPENFOAM

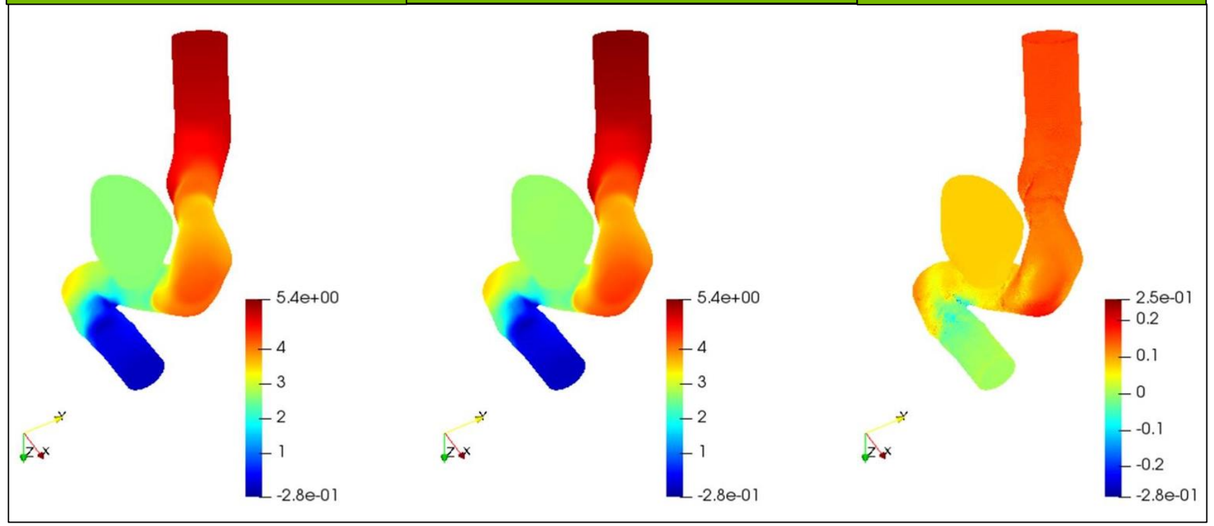
FORWARD SIMULATION

Simnet	OpenFOAM	Difference



Velocities

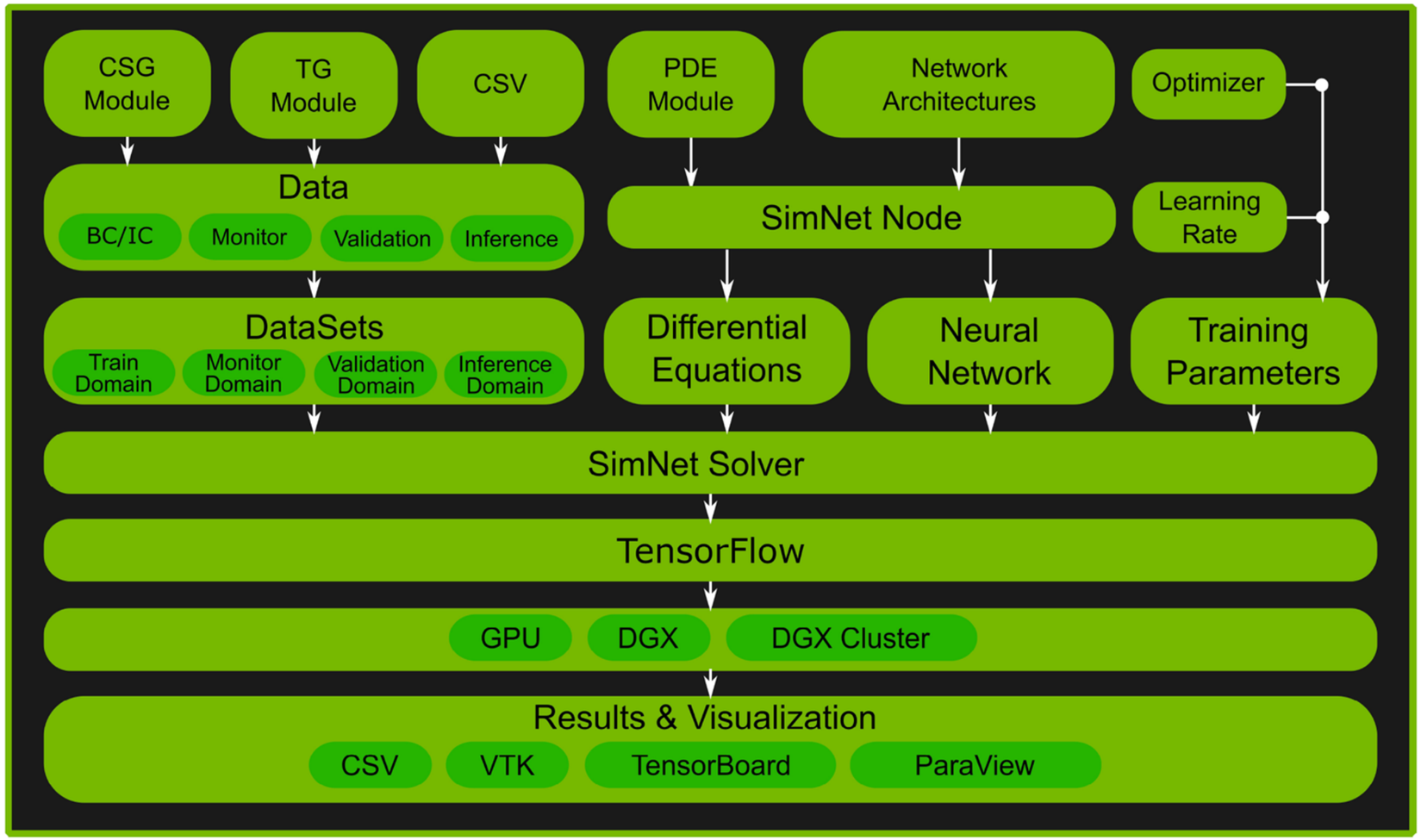
Simnet	OpenFOAM	Difference



Pressures

MODULUS

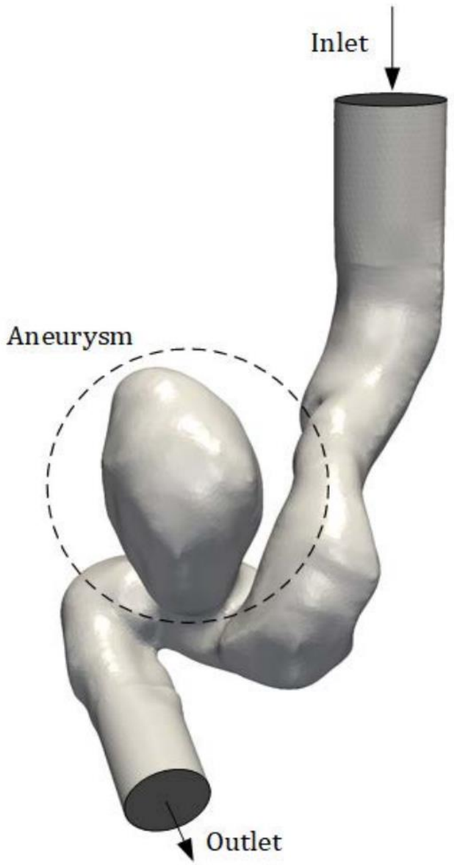
https://developer.nvidia.com/modulus



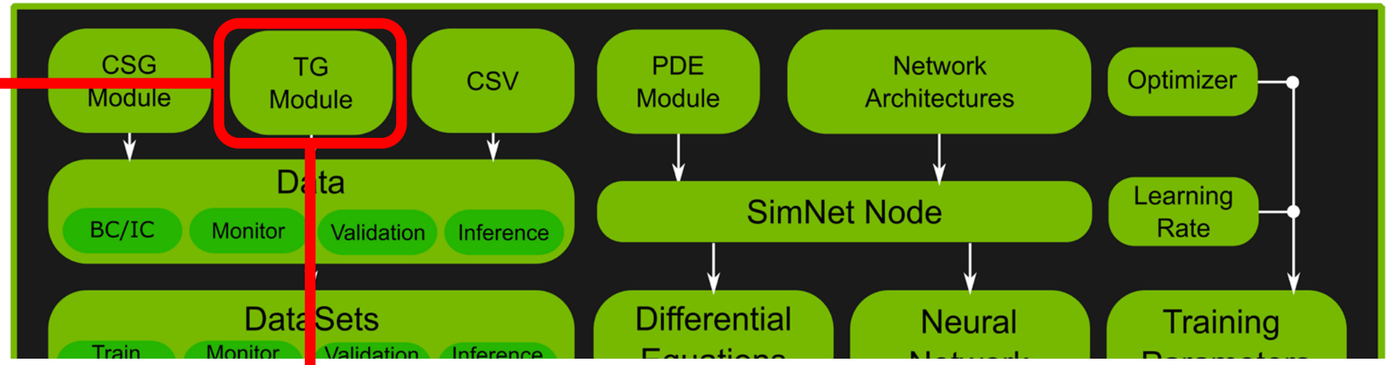
NVIDIA.

EXAMPLE REVIEW

TG Module import STL files



(a) Aneurysm geometry




```
# read stl files to make meshes
point_path = './stl_files/'
inlet_mesh = Mesh.from_stl(point_path + 'aneurysm_inlet.stl', airtight=False)
outlet_mesh = Mesh.from_stl(point_path + 'aneurysm_outlet.stl', airtight=False)
noslip_mesh = Mesh.from_stl(point_path + 'aneurysm_noslip.stl', airtight=False)
integral_mesh = Mesh.from_stl(point_path + 'aneurysm_integral.stl', airtight=False)
interior_mesh = Mesh.from_stl(point_path + 'aneurysm_closed.stl')
```


NVIDIA.

EXAMPLE REVIEW

Implement (Modulus) SimNet Solver

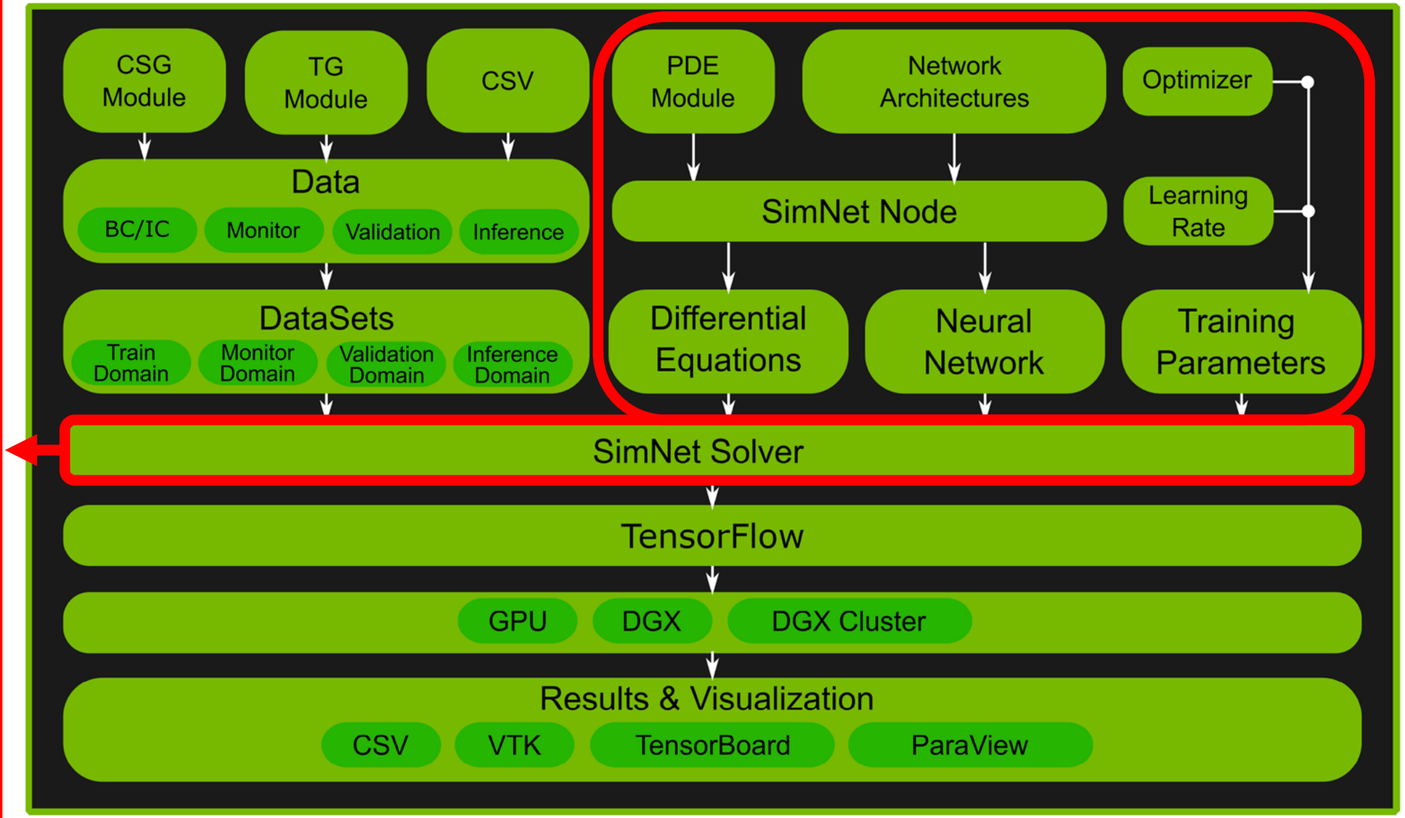
```python
class AneurysmSolver(Solver):
  train_domain = AneurysmTrain
  val_domain = AneurysmVal
  monitor_domain = AneurysmMonitor

  def __init__(self, **config):
    super(AneurysmSolver, self).__init__(**config)
    self.save_network_freq = 20000
    self.print_stats_freq = 500
    self.tf_summary_freq = 500

    self.equations = (NavierStokes(nu=nu*scale, rho=1, dim=3, time=False).make_node()
                      + IntegralContinuity(dim=3).make_node())
    flow_net = self.arch.make_node(name='flow_net',
                                   inputs=['x', 'y', 'z'],
                                   outputs=['u', 'v', 'w', 'p'])
    self.nets = [flow_net]

  @classmethod
  def update_defaults(cls, defaults):
    defaults.update({
        'network_dir': './network_checkpoint_aneurysm',
        'rec_results_cpu': True,
        'max_steps': 100000,
        'decay_steps': 10000,
        'rec_results_freq': 20*1000,
        })

if __name__ == '__main__':
  ctr = SimNetController(AneurysmSolver)
  ctr.run()
```



NVIDIA

EXAMPLE REVIEW

Implement Train Domain

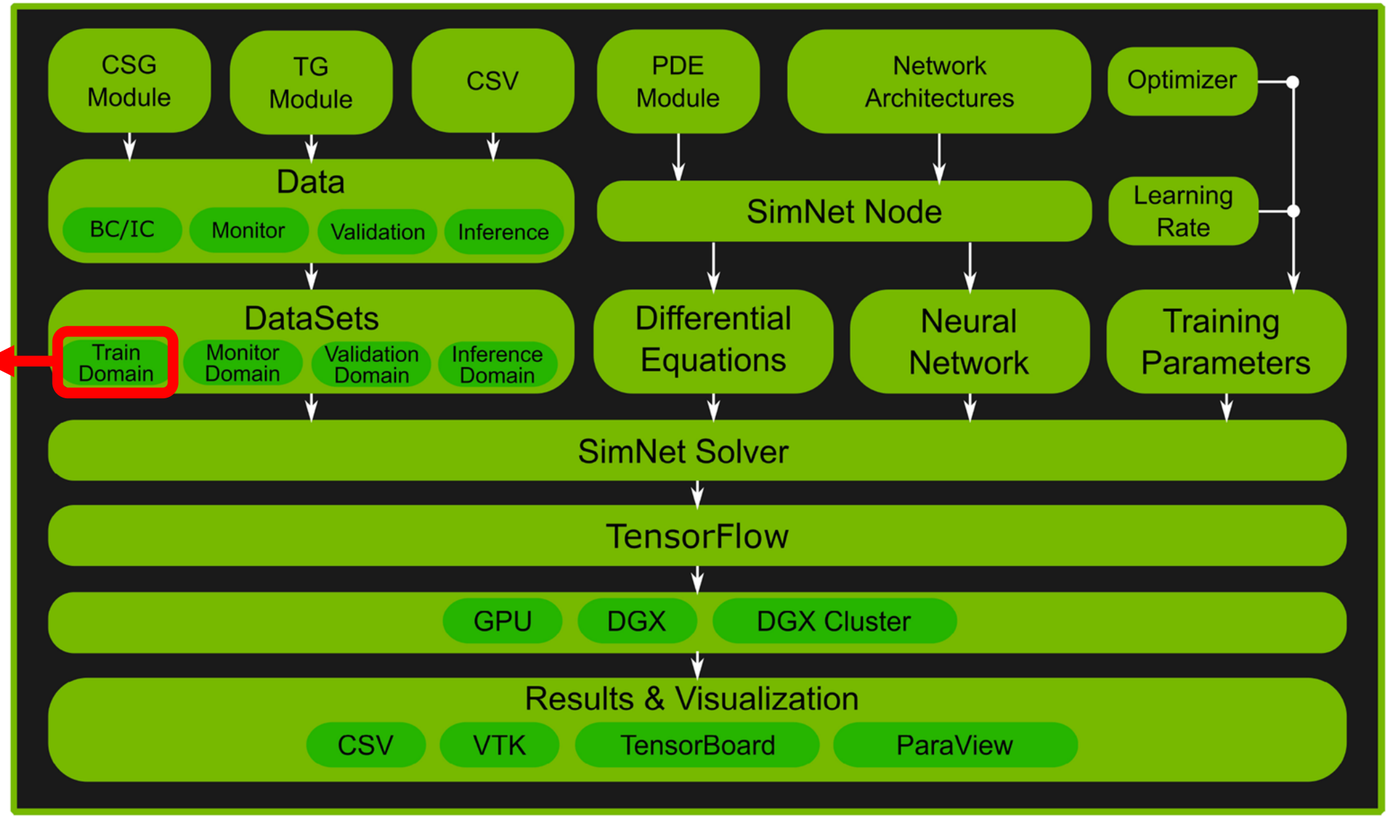
```python
class AneurysmTrain(TrainDomain):
  def __init__(self, **config):
    super(AneurysmTrain, self).__init__()
    # Inlet
    u, v, w = circular_parabola(Symbol('x'),
                                Symbol('y'),
                                Symbol('z'),
                                center=inlet_center,
                                normal=inlet_normal,
                                radius=inlet_radius,
                                max_vel=inlet_vel)
    inlet = inlet_mesh.boundary_bc(outvar_sympy={'u': u, 'v': v, 'w': w},
                                   batch_size_per_area=256)
    self.add(inlet, name="Inlet")

    # Outlet
    outlet = outlet_mesh.boundary_bc(outvar_sympy={'p': 0},
                                     batch_size_per_area=256)
    self.add(outlet, name="Outlet")

    # Noslip
    noslip = noslip_mesh.boundary_bc(outvar_sympy={'u': 0, 'v': 0, 'w': 0},
                                     batch_size_per_area=32)
    self.add(noslip, name="Noslip")
    # Interior
    interior = interior_mesh.interior_bc(outvar_sympy={'continuity': 0,
                                                       'momentum_x': 0,
                                                       'momentum_y': 0,
                                                       'momentum_z': 0},
                                         batch_size_per_area=128,
                                         batch_per_epoch=1000)
    self.add(interior, name="Interior")

    # Integral Continuity 1
    ic_1 = outlet_mesh.boundary_bc(outvar_sympy={'integral_continuity': 2.540},
                                   lambda_sympy={'lambda_integral_continuity': 0.1},
                                   batch_size_per_area=128)
    self.add(ic_1, name="IntegralContinuity_1")

    # Integral Continuity 2
    ic_2 = integral_mesh.boundary_bc(outvar_sympy={'integral_continuity': -2.540},
                                     lambda_sympy={'lambda_integral_continuity': 0.1},
                                     batch_size_per_area=128)
    self.add(ic_2, name="IntegralContinuity_2")
```

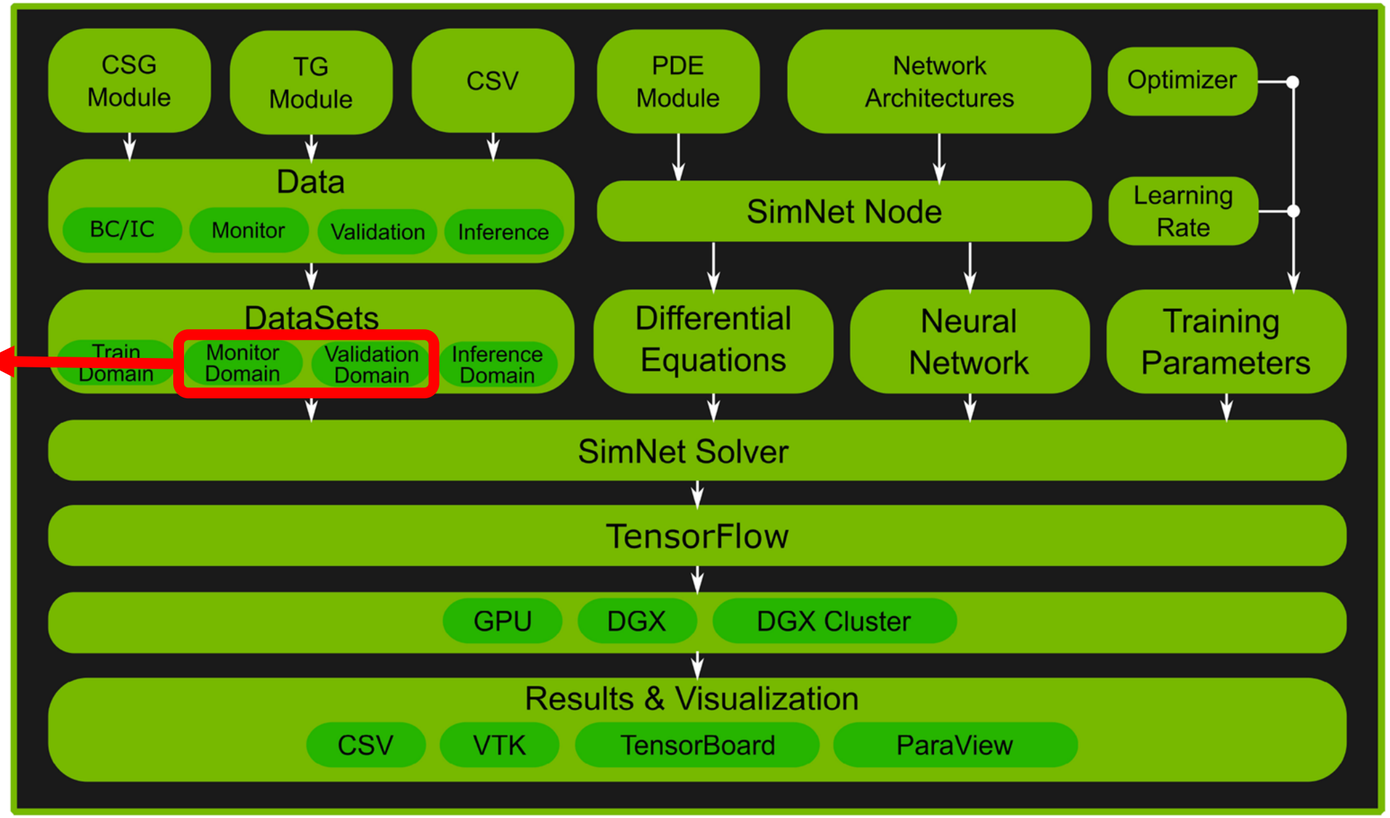


EXAMPLE REVIEW

Implement Monitor/Validation Domain

```python
class AneurysmVal(ValidationDomain):
  def __init__(self, **config):
    super(AneurysmVal, self).__init__()
    val = Validation.from_numpy(openfoam_invar, openfoam_outvar)
    self.add(val, name='Val')

class AneurysmMonitor(MonitorDomain):
  def __init__(self, **config):
    super(AneurysmMonitor, self).__init__()
    # metric for pressure drop
    metric = Monitor(inlet_mesh.sample_boundary(16),
                     {'pressure_drop': lambda var: tf.reduce_mean(var['p'])})
    self.add(metric, 'PressureDrop')
```

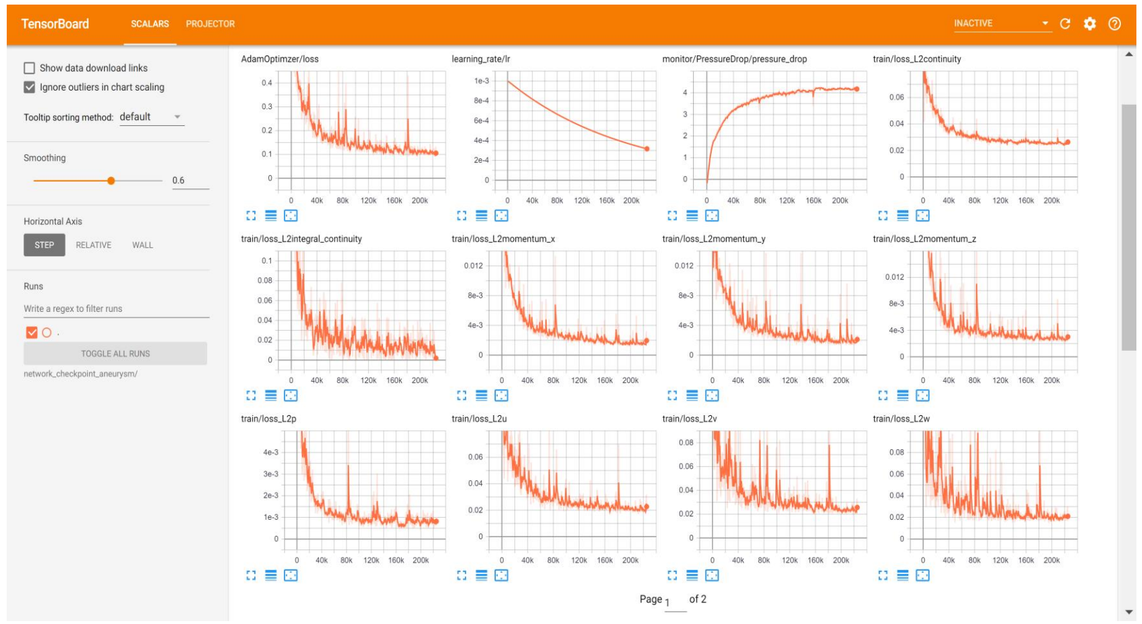


NVIDIA.

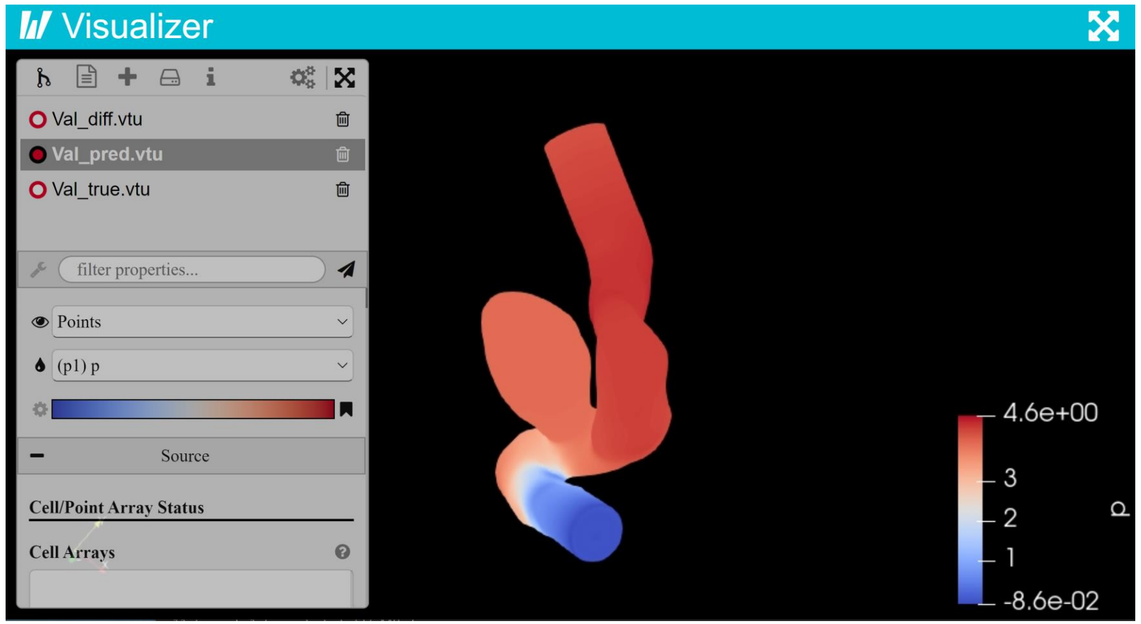
EXAMPLE REVIEW

Start Training by command “**python aneurysm.py**”
(Monitoring by TensorBoard, Visualizing by ParaView)

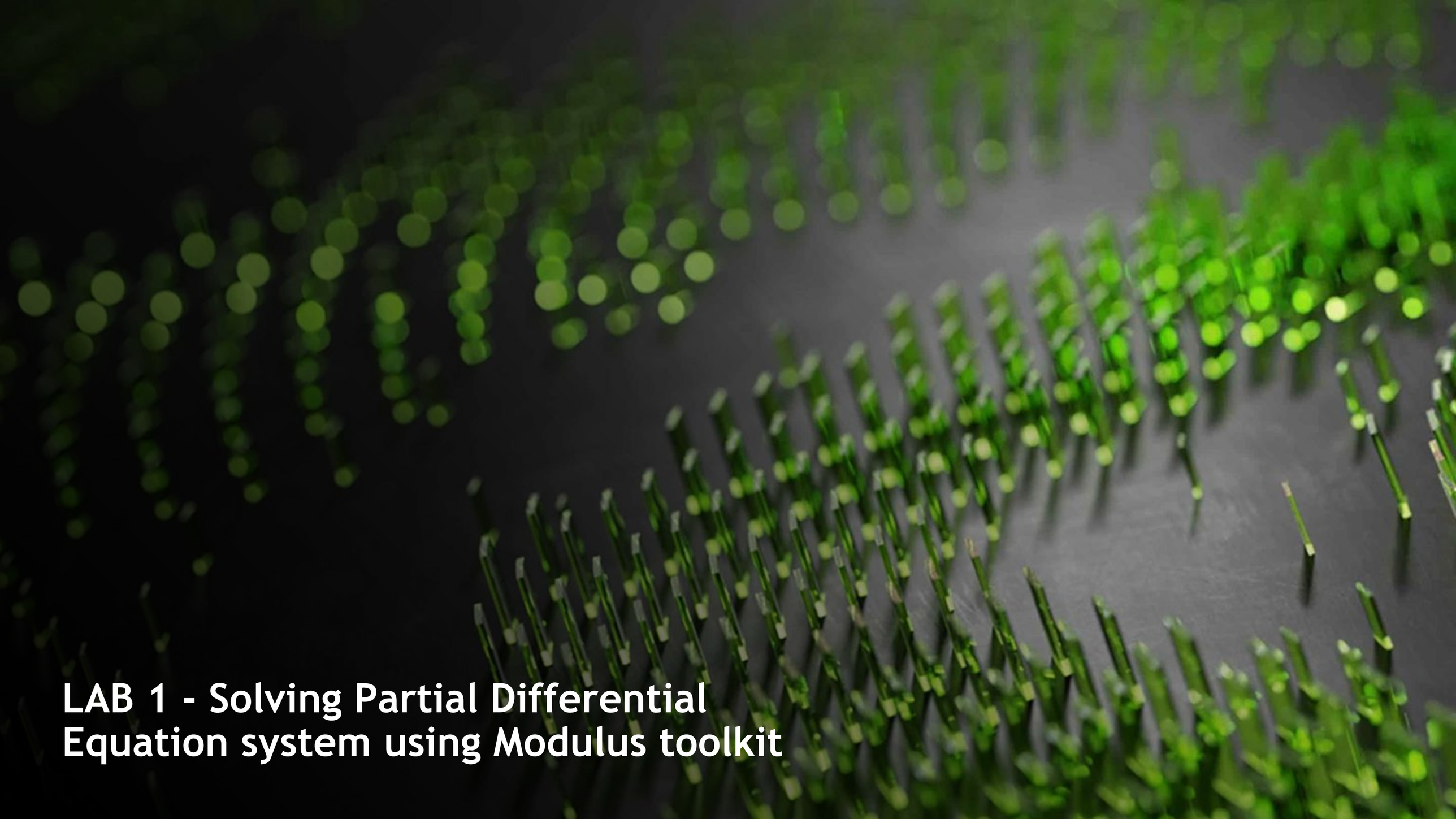
NVIDIA SimNet Tutorial - Simulate Blood Flow in Intracranial Aneurysm (Chinese)



TensorBoard

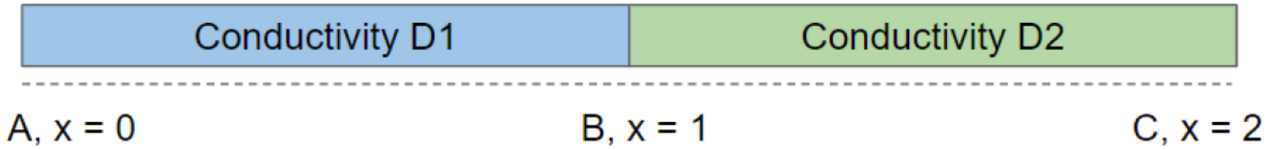


Paraview



LAB 1 - Solving Partial Differential Equation system using Modulus toolkit

SOLUTION TO PDES- 1D DIFFUSION



- Composite bar with material of conductivity $D_1 = 10$ for $x \in (0,1)$ and $D_2 = 0.1$ for $x \in (1,2)$. Point A and C are maintained at temperatures of 0 and 100 respectively

- Equations: Diffusion equation in 1D

$$\frac{d}{dx}\left(D_1 \frac{dU_1}{dx}\right) = 0 \qquad \text{When } 0 < x < 1$$

$$\frac{d}{dx}\left(D_2 \frac{dU_2}{dx}\right) = 0 \qquad \text{When } 1 < x < 2$$

- Flux and field continuity at interface $(x = 1)$

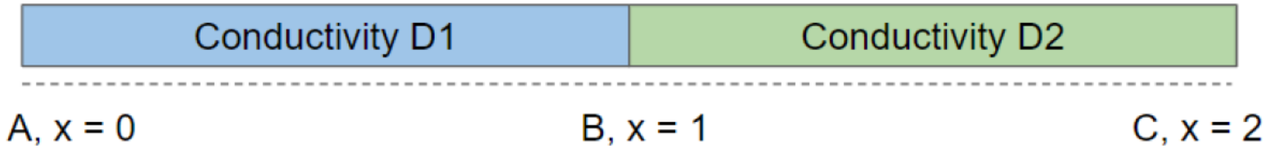
$$\left(D_1 \frac{dU_1}{dx}\right) = \left(D_2 \frac{dU_2}{dx}\right)$$

$$U_1 = U_2$$

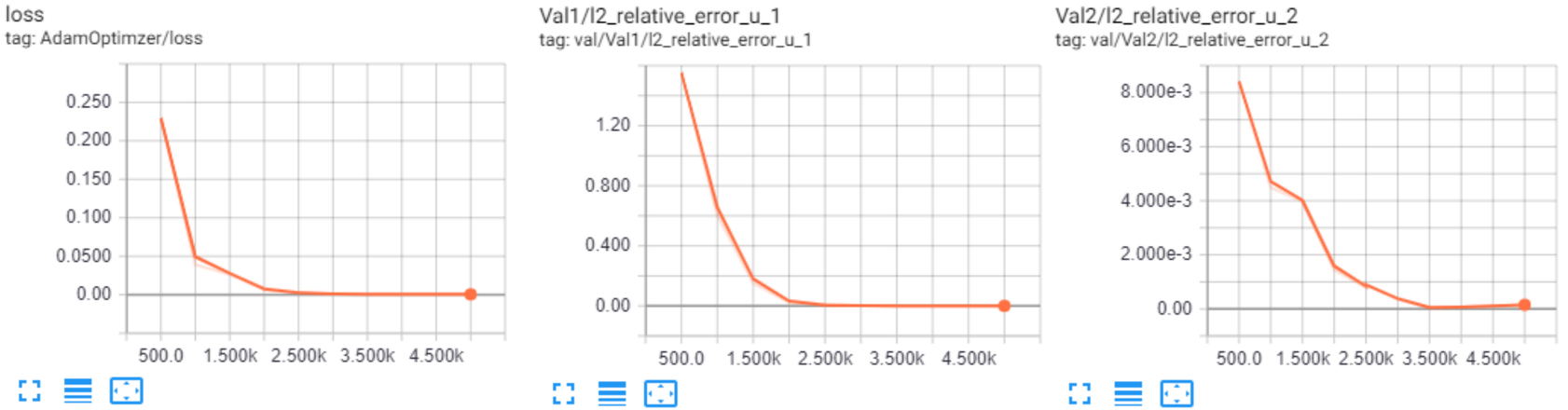
This material is released by NVIDIA Corporation under the Creative Commons Attribution 4.0 International (CC BY 4.0)

NVIDIA.

SOLUTION TO PDES- 1D DIFFUSION

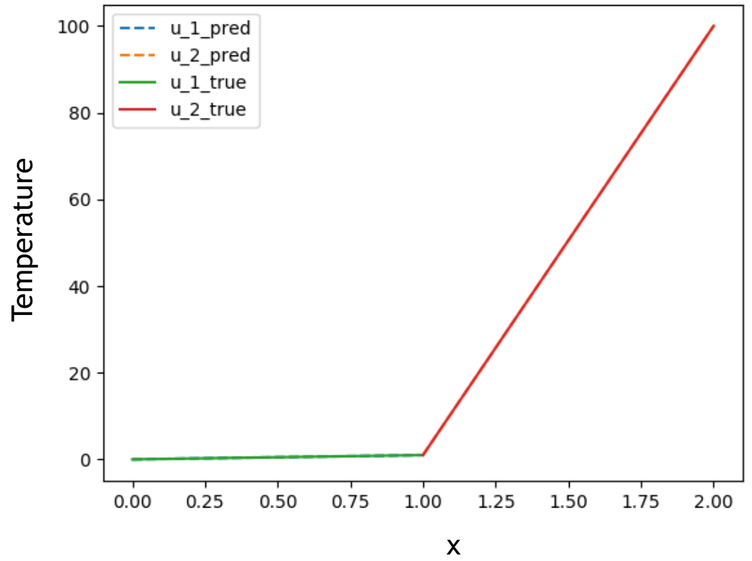


- Define the problem and train the neural network to obtain the temperature distribution in the bar
- Compare the results with analytical solution

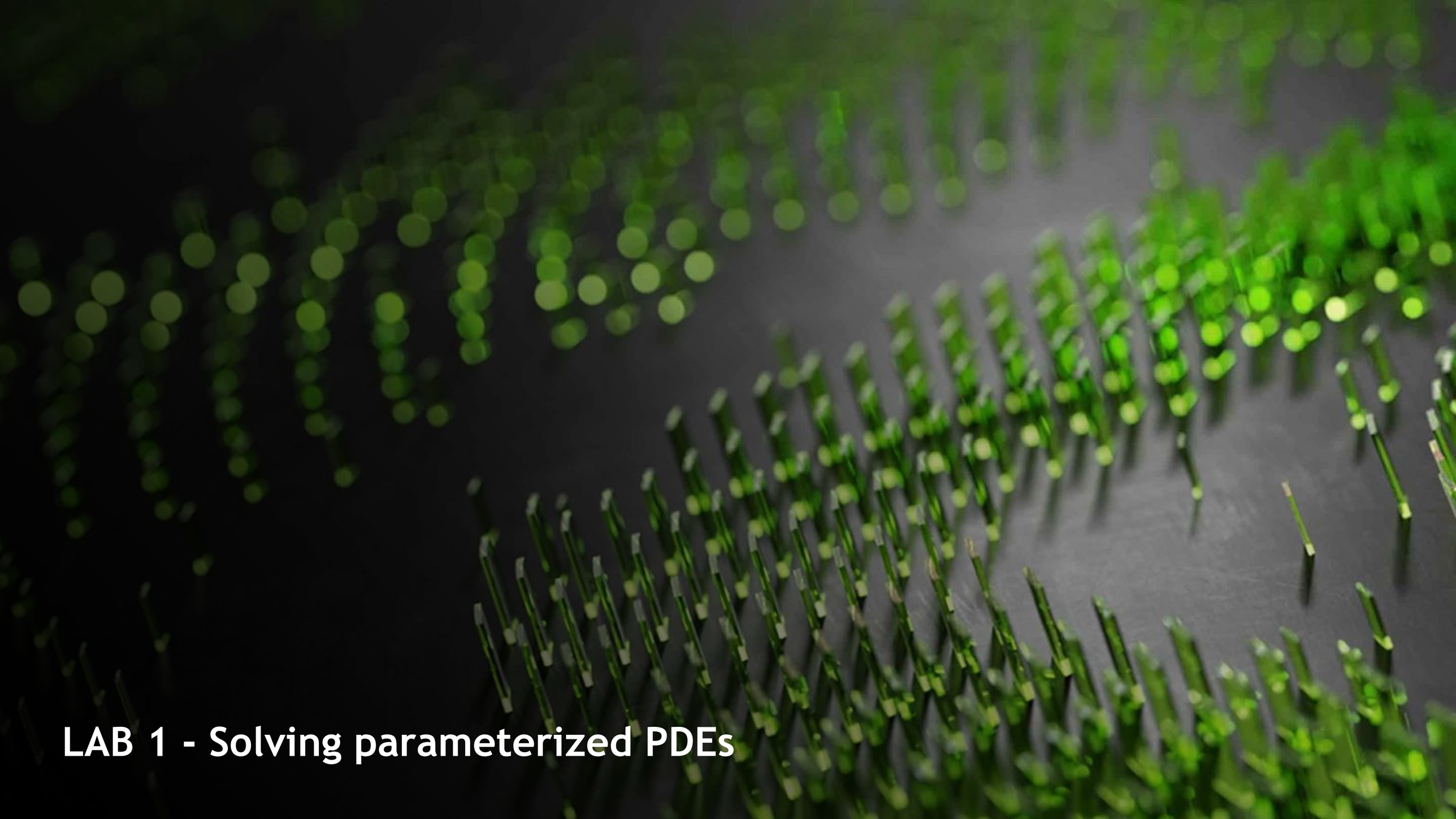


Training error

Validation error

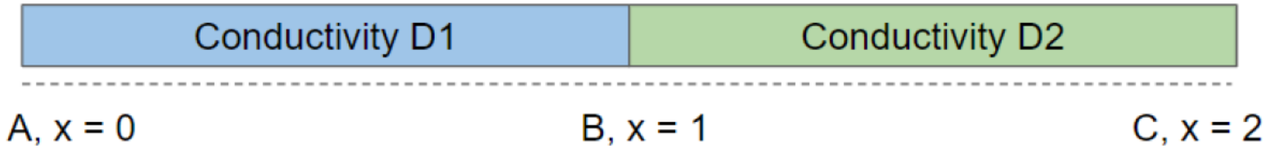


This material is released by NVIDIA Corporation under the Creative Commons Attribution 4.0 International (CC BY 4.0)

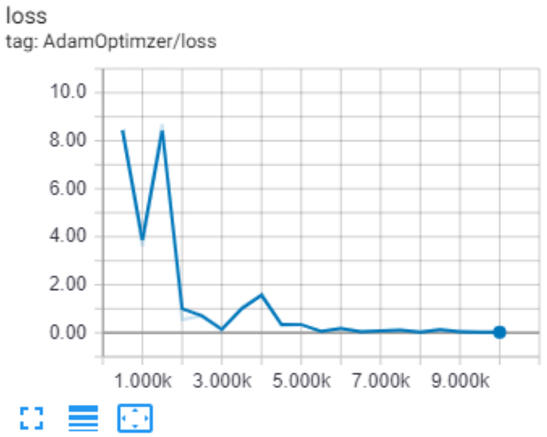


LAB 1 - Solving parameterized PDEs

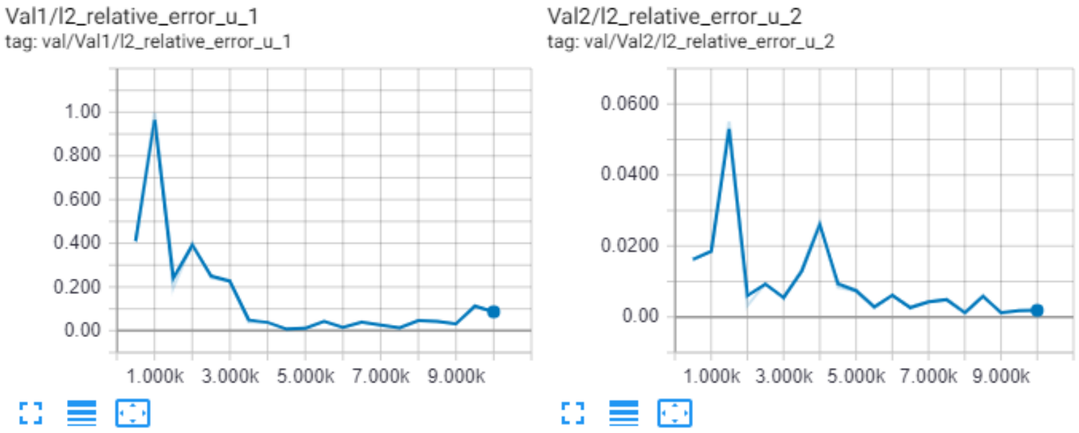
SOLUTION TO PARAMETERIZED PDES- 1D DIFFUSION



- Composite bar with material of conductivity D_1 for $x \in (0,1)$ and $D_2 = 0.1$ for $x \in (1,2)$.
- Solve the problem for multiple values of D_1 in the range $(5, 25)$ in a single training
- Same boundary and interface conditions as before



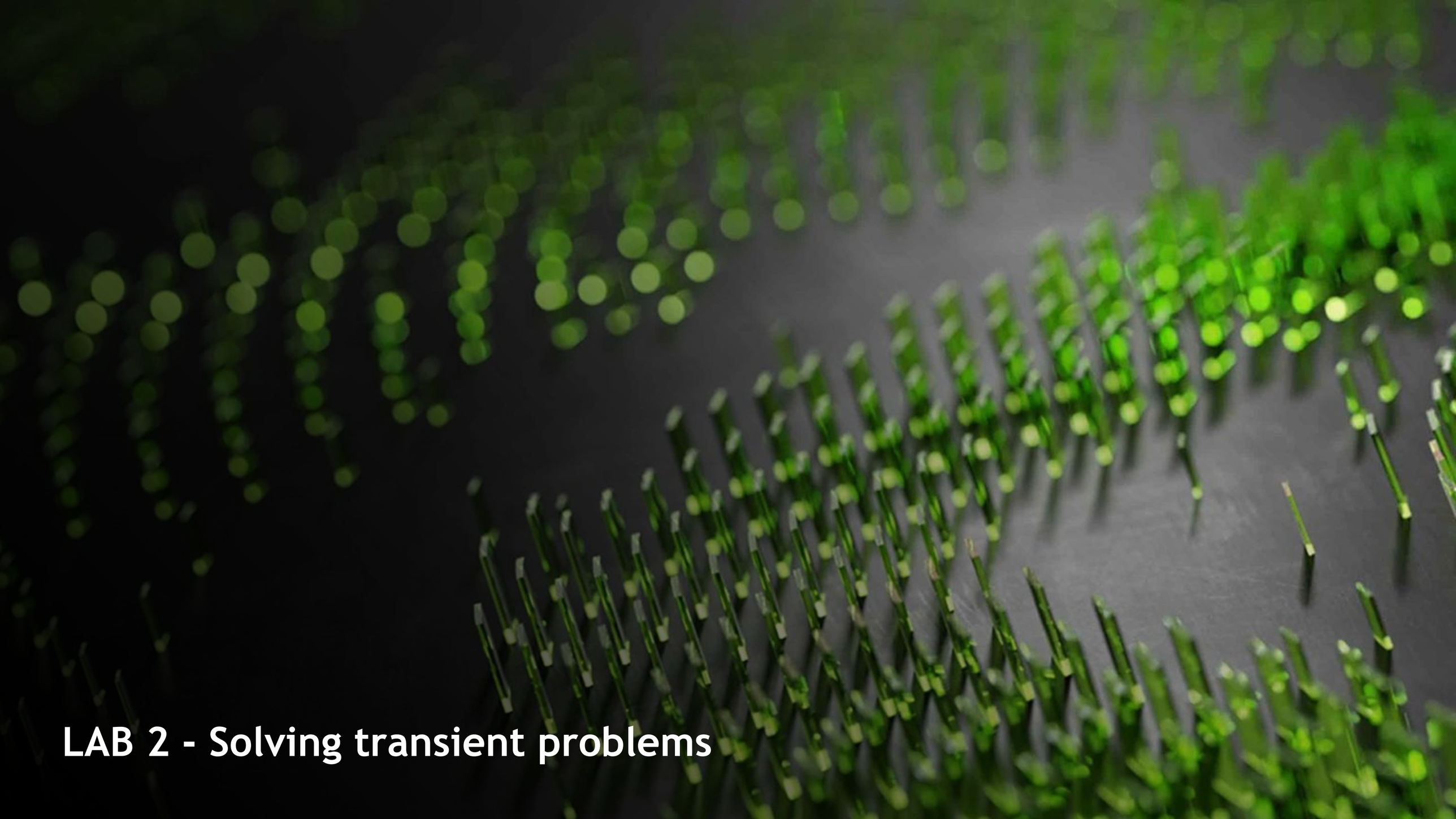
Training error



Validation error for $D_1 = 10$

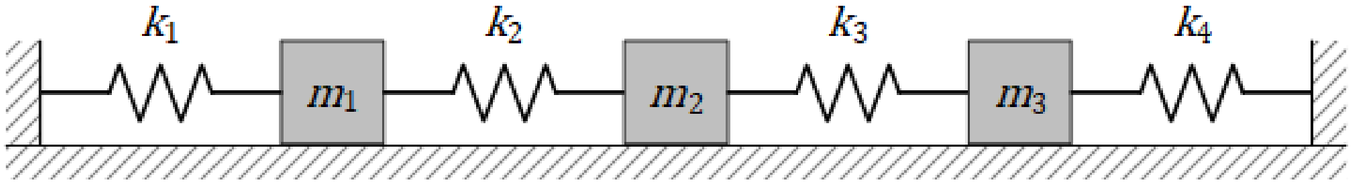
This material is released by NVIDIA Corporation under the Creative Commons Attribution 4.0 International (CC BY 4.0)

NVIDIA.



LAB 2 - Solving transient problems

SOLUTION TO ODES- COUPLED SPRING MASS SYSTEM



- Three masses connected by four springs
- System's equations (ordinary differential equations):

$$m_1 x_1''(t) = -k_1 x_1(t) + k_2\big(x_2(t) - x_1(t)\big)$$
$$m_2 x_2''(t) = -k_2\big(x_2(t) - x_1(t)\big) + k_3\big(x_3(t) - x_2(t)\big)$$
$$m_3 x_3''(t) = -k_3\big(x_3(t) - x_2(t)\big) - k_4 x_3(t)$$

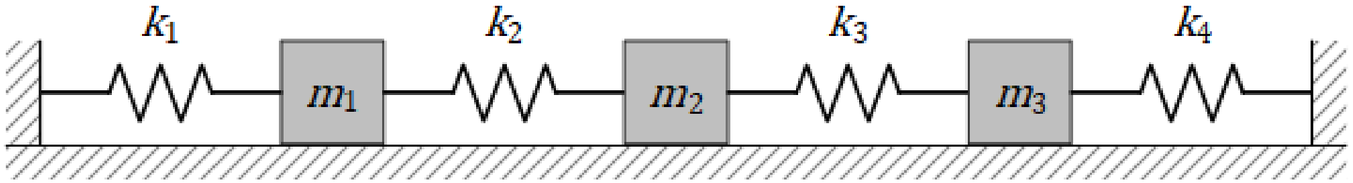
- For given values masses, spring constants and boundary conditions

$$[m_1, m_2, m_3] = [1, 1, 1]$$
$$[k_1, k_2, k_3, k_4] = [2, 1, 1, 2]$$
$$[x_1(0), x_2(0), x_3(0)] = [1, 0, 0]$$
$$[x_1'(0), x_2'(0), x_3'(0)] = [0, 0, 0]$$

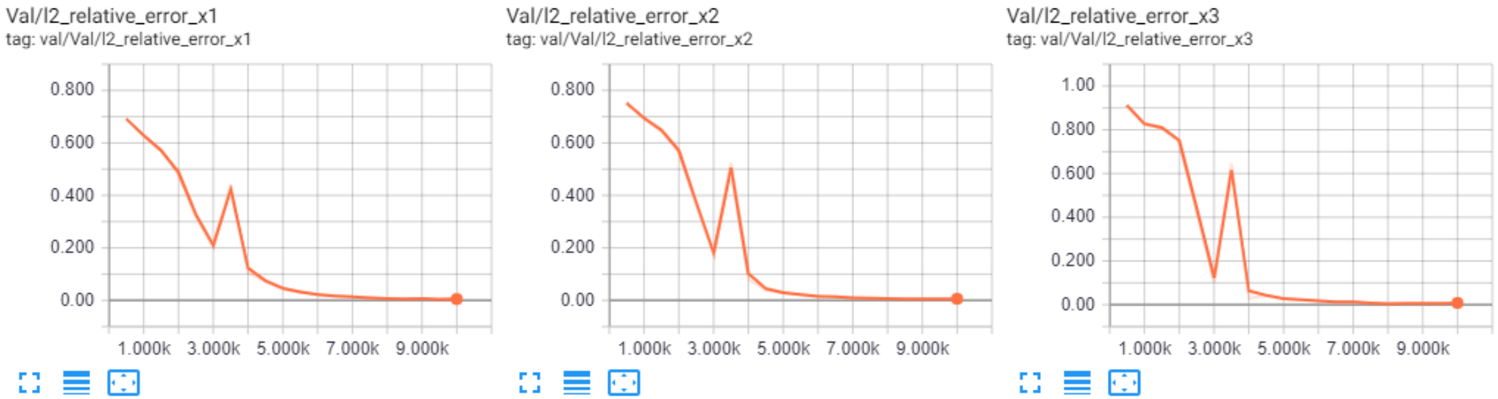
This material is released by NVIDIA Corporation under the Creative Commons Attribution 4.0 International (CC BY 4.0)

NVIDIA.

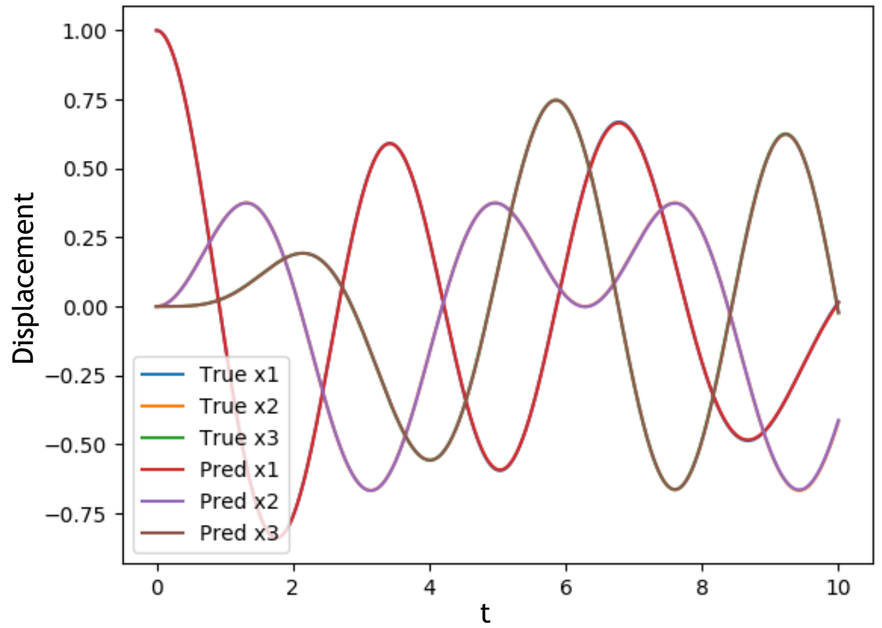
SOLUTION TO ODES- COUPLED SPRING MASS SYSTEM



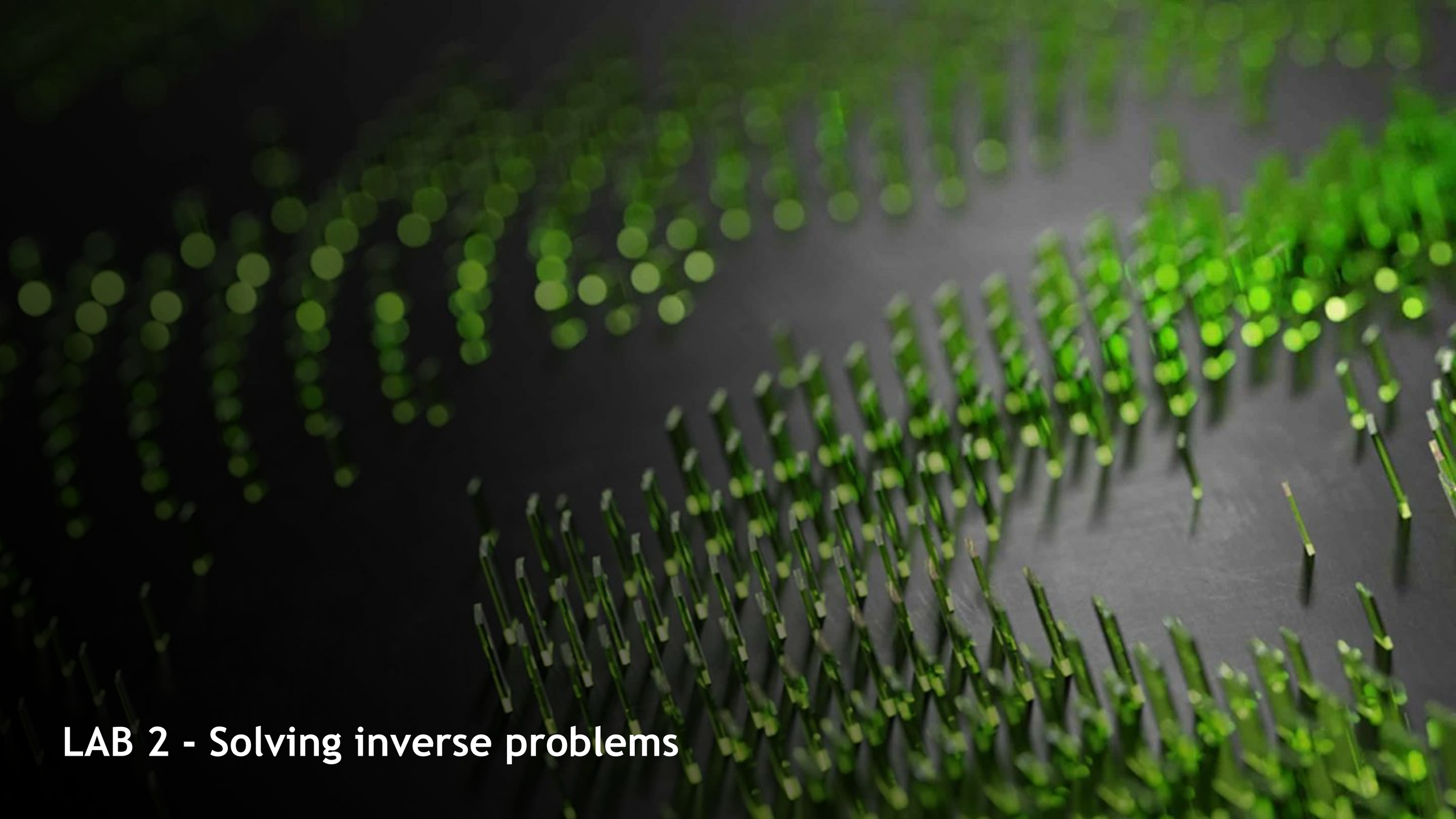
- Define the transient problem for time, $t = (0, 10)$ and train the neural network to obtain the displacement of each mass
- Compare the results with analytical solution



Validation error for each mass

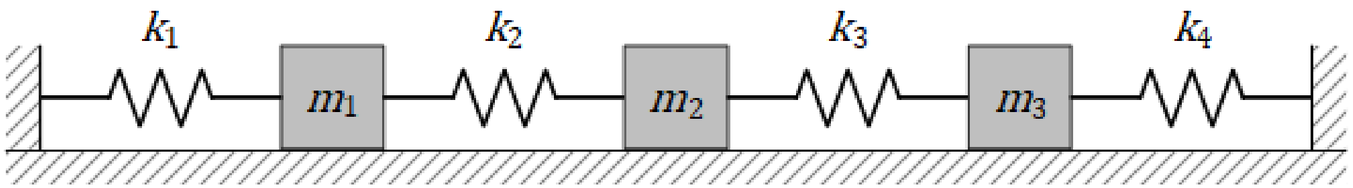


This material is released by NVIDIA Corporation under the Creative Commons Attribution 4.0 International (CC BY 4.0)



LAB 2 - Solving inverse problems

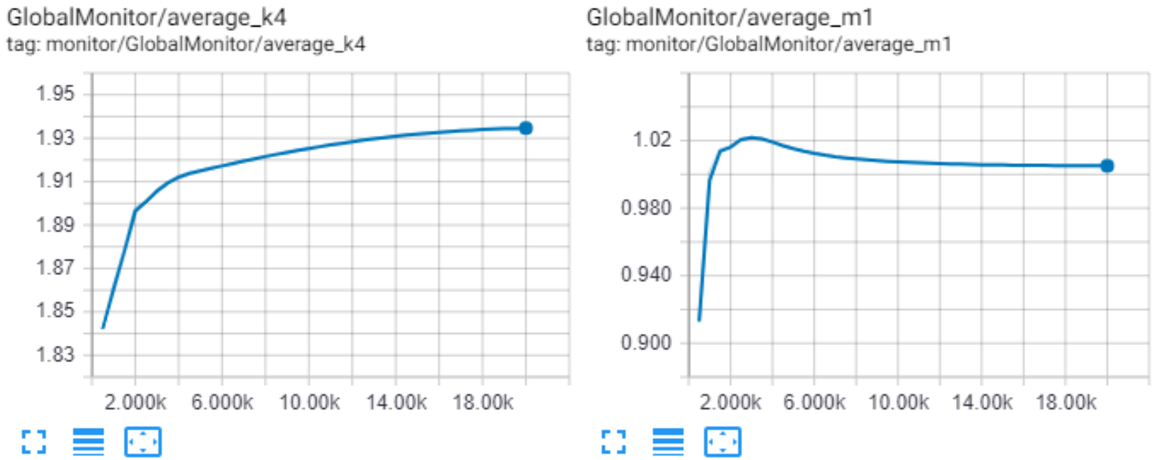
INVERSE PROBLEMS- COUPLED SPRING MASS SYSTEM



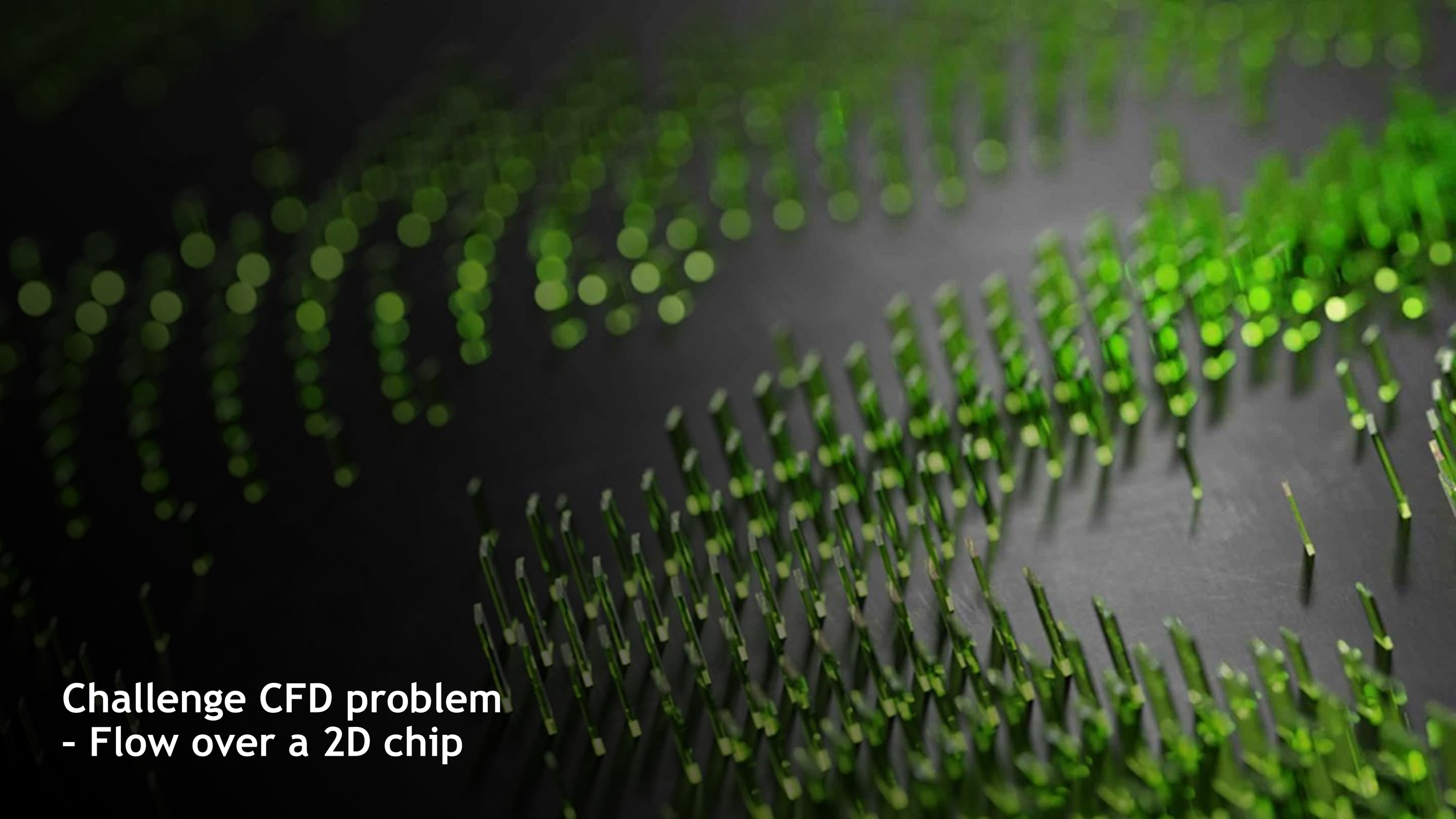
- For the same system, assume we know the analytical solution which is given by:

$$x_1(t) = \frac{1}{6}\cos(t) + \frac{1}{2}\cos\left(\sqrt{3}t\right) + \frac{1}{3}\cos(2t), \qquad x_2(t) = \frac{2}{6}\cos(t) - \frac{1}{3}\cos(2t), \qquad x_3(t) = \frac{1}{6}\cos(t) - \frac{1}{2}\cos\left(\sqrt{3}t\right) + \frac{1}{3}\cos(2t)$$

- With the above data and the values for m_2, m_3, k_1, k_2, k_3 same as before, use the neural network to find the values of m_1 and k_4

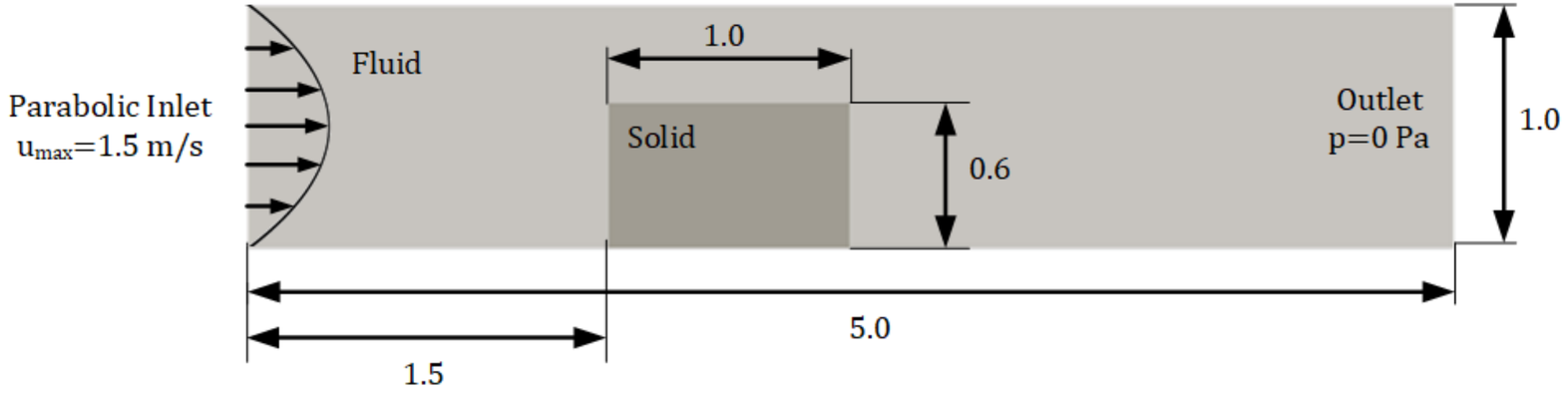


This material is released by NVIDIA Corporation under the Creative Commons Attribution 4.0 International (CC BY 4.0)



Challenge CFD problem
- Flow over a 2D chip

CHALLENGE: FLOW OVER 2D CHIP



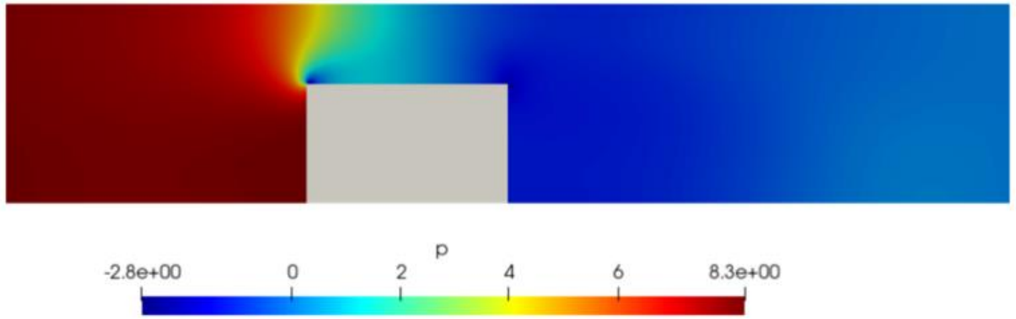
- Solve the flow over 2D chip for the given boundary conditions. The challenge problem has 3 parts:

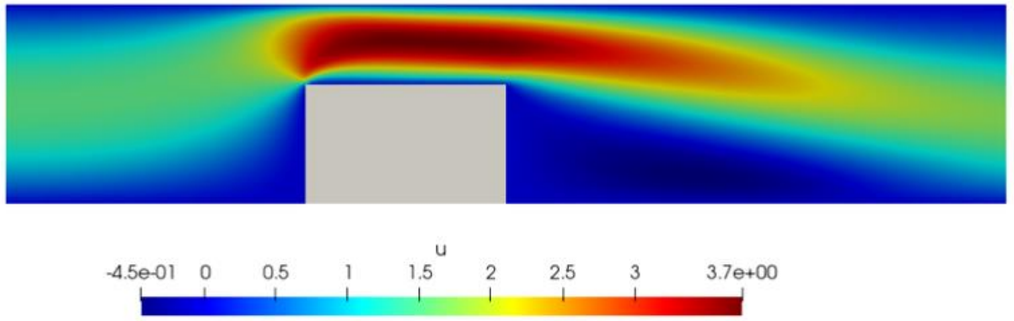
1. Solve the fluid flow for the given boundary conditions and geometry
2. Solve the fluid flow for the parameterized Chip geometry
3. Solve the inverse problem where, given a flow field, use it to invert out the viscosity of the flow

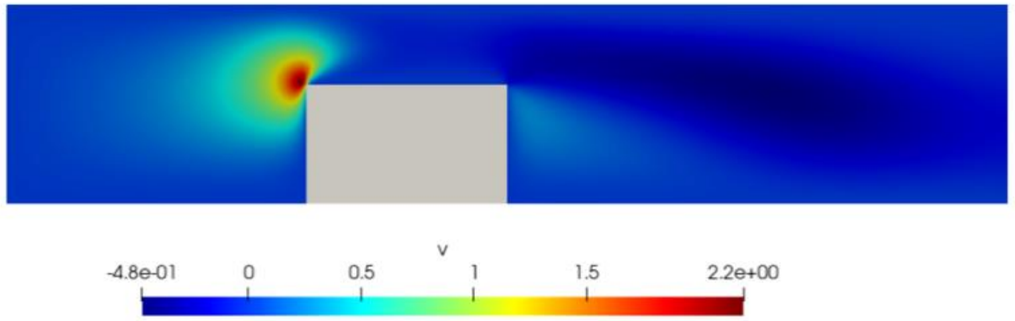
This material is released by NVIDIA Corporation under the Creative Commons Attribution 4.0 International (CC BY 4.0)

CHALLENGE: FLOW OVER 2D CHIP

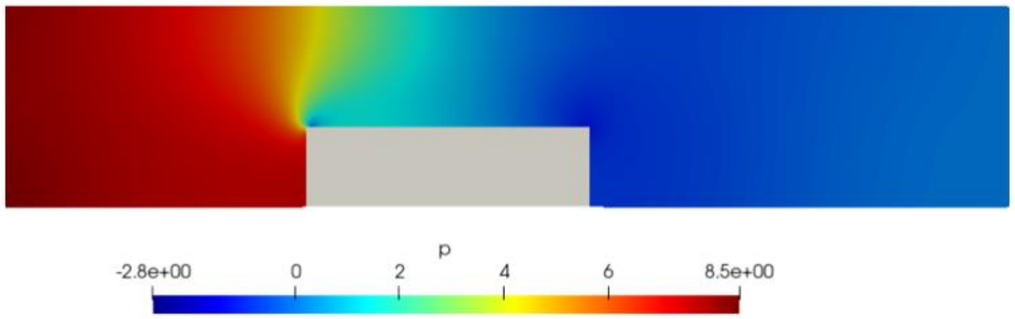
h: 0.6, w: 1.0

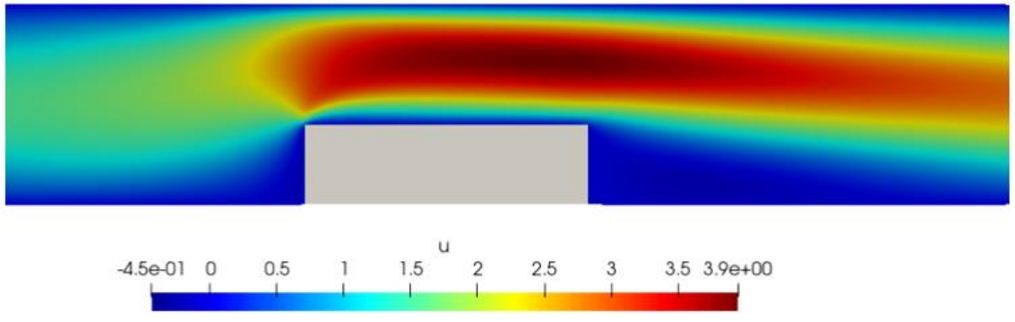


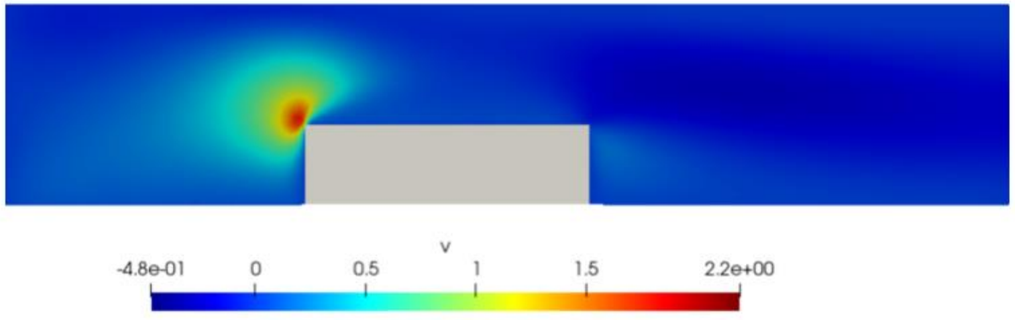




h: 0.4, w: 1.4







This material is released by NVIDIA Corporation under the Creative Commons Attribution 4.0 International (CC BY 4.0)

FOURIER FEATURES LET NETWORKS LEARN HIGH FREQUENCY FUNCTIONS IN LOW DIMENSIONAL DOMAINS

https://bmild.github.io/fourfeat/

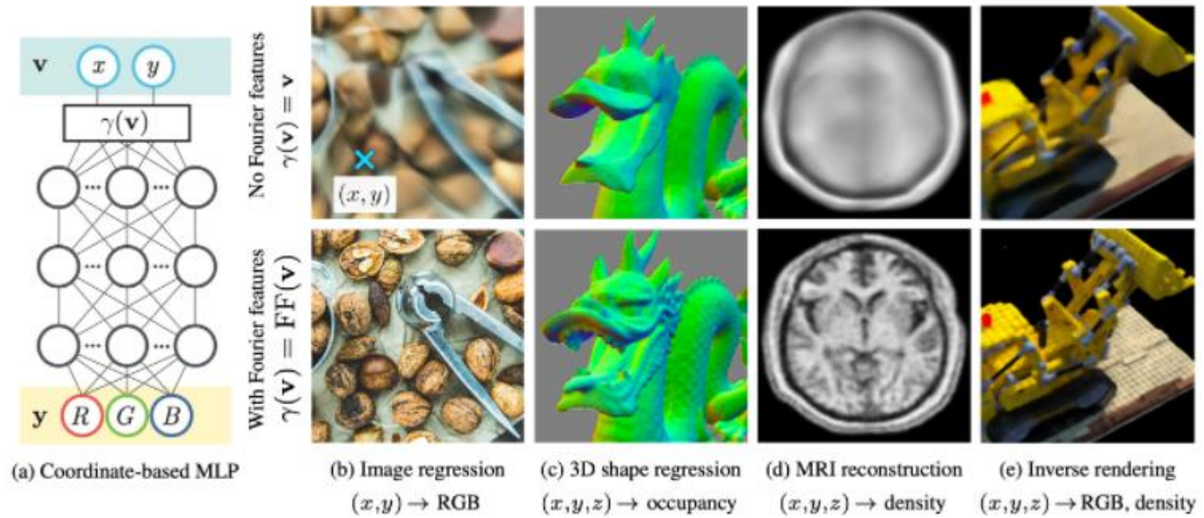
Matthew Tancik* Pratul Srinivasan* Ben Mildenhall* Sara Fridovich-Keil
UC Berkeley UC Berkeley UC Berkeley UC Berkeley

Nithin Raghavan Utkarsh Singhal Ravi Ramamoorthi Jonathan T. Barron Ren Ng
UC Berkeley UC Berkeley UC San Diego Google Research UC Berkeley

*denotes equal contribution

Paper Code

Abstract



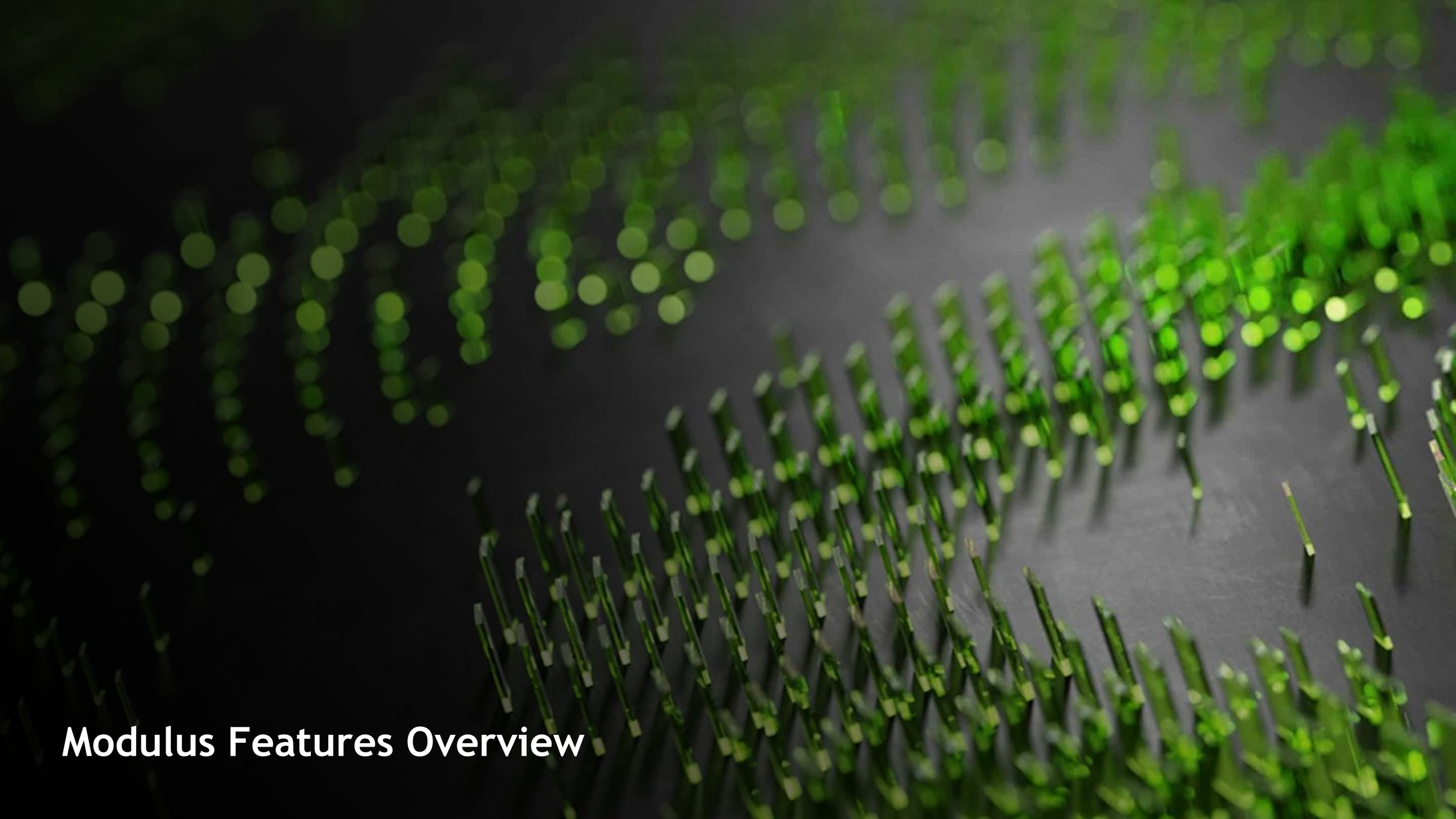
(a) Coordinate-based MLP (b) Image regression $(x,y) \rightarrow$ RGB (c) 3D shape regression $(x,y,z) \rightarrow$ occupancy (d) MRI reconstruction $(x,y,z) \rightarrow$ density (e) Inverse rendering $(x,y,z) \rightarrow$ RGB, density

NVIDIA.

CHALLENGE: FLOW OVER 2D CHIP- HINTS AND TIPS

- Use Signed Distance Function to weight the equation losses inside domain for faster convergence (User Guide Section 2.3.2)
- Use Integral Continuity for faster convergence (User Guide Section 8.3.1 and 8.3.2)

This material is released by NVIDIA Corporation under the Creative Commons Attribution 4.0 International (CC BY 4.0)

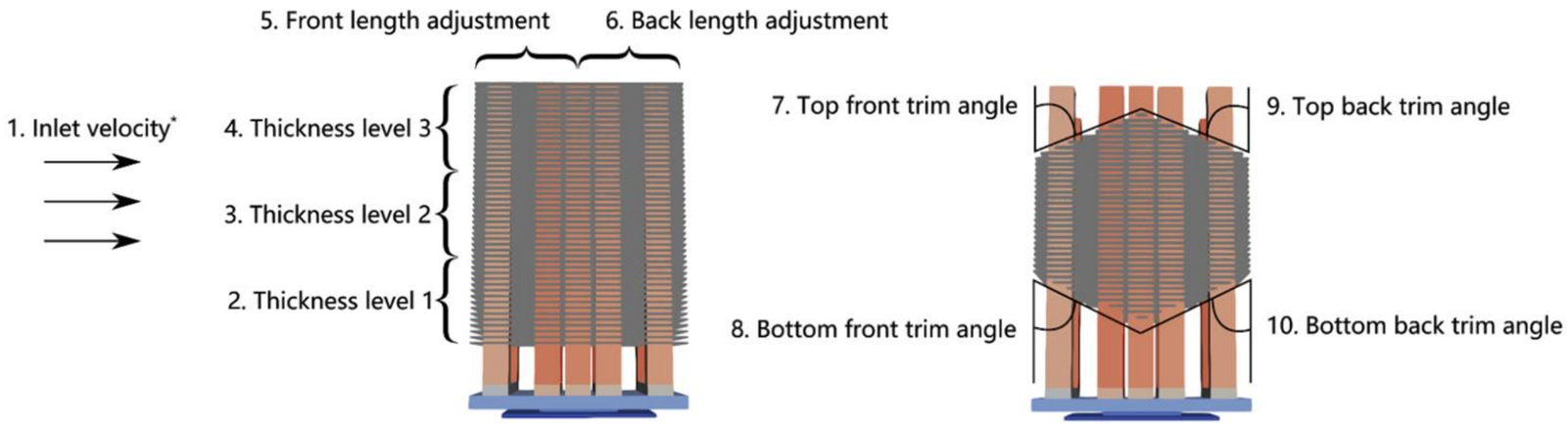


Modulus Features Overview

PARAMETERIZED A100 NVSWITCH HEAT SINK

Optimization with 10 Design Parameters

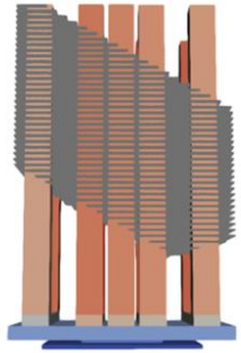
Limerock Design Variables



*Inlet velocity is not a design variable. It will be used for robust design optimization of Limerock in future.

Limerock Optimal Design

1. Inlet velocity: 5.7 m/s
2. Thickness level 1: 0.0031
3. Thickness level 2: 0.0044
4. Thickness level 3: 0.0030
5. Front length adjustment: -0.0124
6. Back length adjustment: 0.0025
7. Top front trim angle: 0.0223 rad
8. Bottom front trim angle: 0.5197 rad
9. Top back trim angle: 0.5147 rad
10. Bottom back trim angle: 0.2217 rad

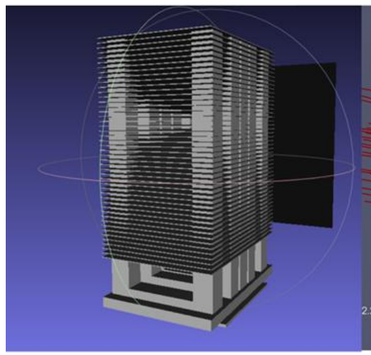


Max allowed pressure drop: 2.59
Number of random design evaluations: 4M

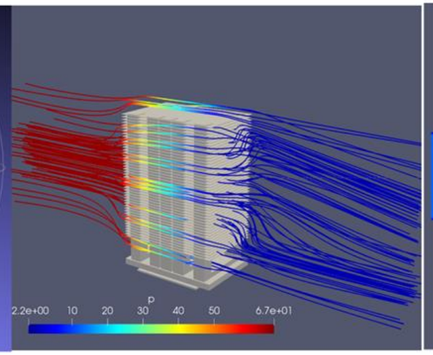
Peak temperature: 38.25 degC
Pressure drop: 2.5896

A100 NVSWITCH HEAT SINK

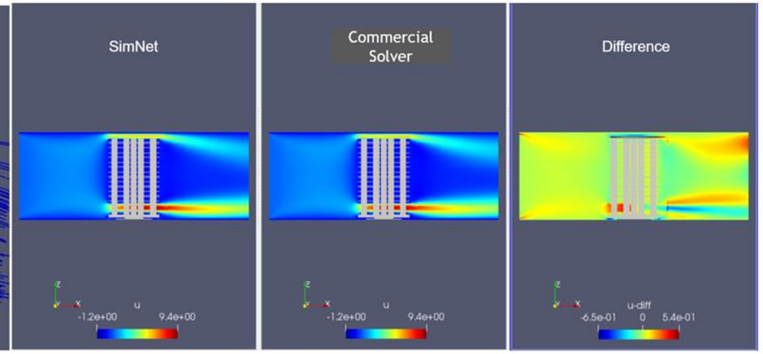
Multi-Physics Application: Fluids + Heat Transfer



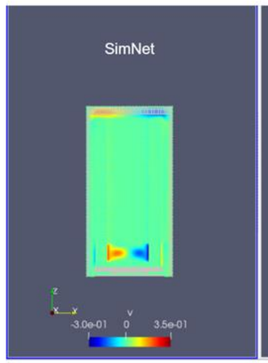
Nvidia DGX A100 Heatsink

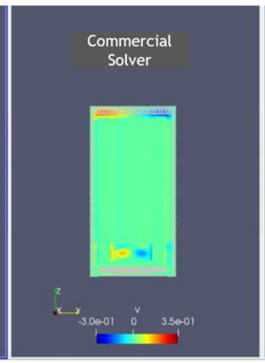


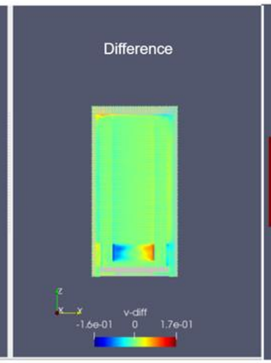
Pressure color coded flow streamlines



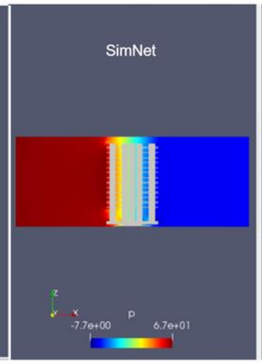
U-velocity (transverse direction) comparison

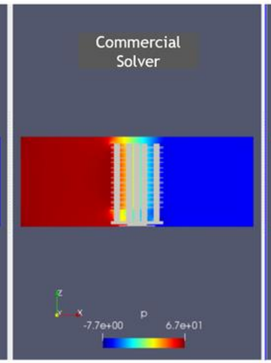


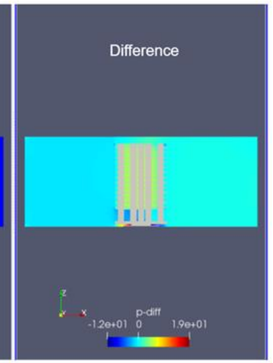




V-velocity (transverse direction) comparison







Pressure comparison

Turbulent Flow (Re=19,000)		
	Temperature	**Pressure Drop**
SimNet - Fourier Network	**43.1°C**	**4.05**
OpenFOAM (method 1)	**41.6 °C**	**3.56**
OpenFOAM (method 2)	**41.6 °C**	**4.58**

Computational Times (10 parameters, 3 values per parameter)	
SimNet	**1000 V100 GPU hrs.**
~=18000x speedup	
Traditional Solver (OpenFOAM) 59,049 separate runs (26 wall hours on 12 CPU cores)	**18.4 Million CPU hrs.**

NVIDIA.

IMPROVED PREDICTIONS WITH MODULUS

Solution Accuracy Studies

Just channel with no heatsink

Mesh Size and details	Elements	Modulus P drop	Commercial Solver P drop	% diff.
Base mesh	8.5 M	4.45	3.80	17.04
1.3x refinement	14 M	4.45	3.88	14.64
2.7x refinement	23 M	4.45	3.89	14.36
1.2x refinement + Wall refinement	10 M	4.45	4.32	2.95
2.4x refinement + Wall refinement	20 M	4.45	4.40	1.23

FPGA heatsink

Mesh Size and details	Elements	Modulus P drop	Commercial Solver P drop	% diff.
Base mesh + Wall refinement	15 M	27.45	25.23	8.81
1.7x refinement + Wall refinement	25 M	27.45	26.16	4.94

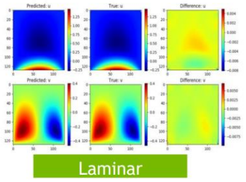
NVIDIA.

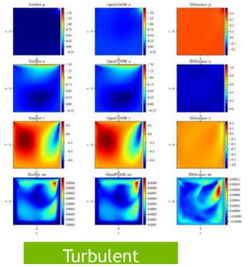
MODULUS FEATURES AND ADVANCEMENTS

Physics types:
- Linear Elasticity (plane stress, plane strain and 3D)
- Fluid Mechanics
- Heat Transfer
- Coupled Fluid-Thermal
- Electromagnetics
- 2D wave propagation

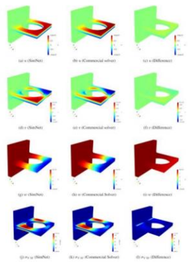
Solution of differential equations:
- Ordinary Differential Equations
- Partial Differential Equations
- Differential (strong) Form
- Integral (weak) form of the PDEs

CFD





Solid Mechanics

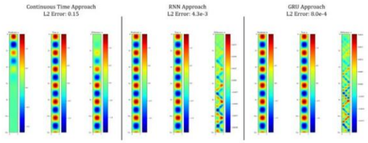


Equilibrium: $\sigma_{ji,j} + f_i = 0,$

Stress-Strain: $\sigma_{ij} = \lambda \epsilon_{kk} \delta_{ij} + 2\mu\epsilon_{ij},$

Strain-Displacement: $\epsilon_{ij} = \frac{1}{2}(u_{i,j} + u_{j,i}).$

Acoustics



$$u_{tt} = c^2 u_{xx}$$
$$u(0,t) = 0,$$
$$u(\pi,t) = 0,$$
$$u(x,0) = \sin(x),$$
$$u_t(x,0) = \sin(x).$$

Electromagnetics

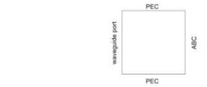


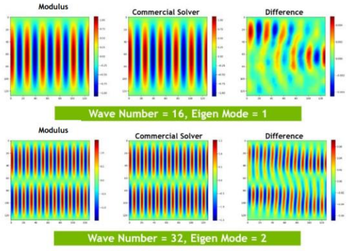
Figure 42: Domain of 2D waveguide

In this example we will solve this waveguide problem by transverse-magnetic (TM_z) mode, so that our unknown variable is $E_z(x, y)$. The governing equation in Ω is

$$\Delta E_z(x,y) + k^2 E_z(x,y) = 0,$$

where k is the wavenumber. Notice in 2D scalar case, the PEC and ABC will be simplified in the following form, respectively:

$E_z(x, y) = 0$ on top and bottom boundaries, $\frac{\partial E_z}{\partial y} = 0$ on right boundary.

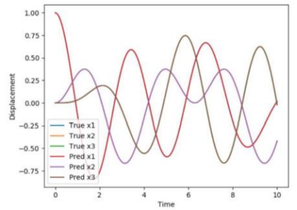


Vibrations

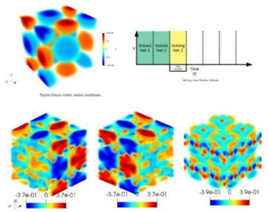
$$m_1 x_1''(t) = -k_1 x_1(t) + k_2(x_2(t) - x_1(t)),$$
$$m_2 x_2''(t) = -k_2(x_2(t) - x_1(t)) + k_3(x_3(t) - x_2(t)),$$
$$m_3 x_3''(t) = -k_3(x_3(t) - x_2(t)) - k_4 x_3(t).$$

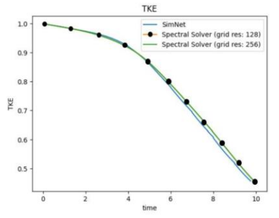
$$[m_1, m_2, m_3] = [1, 1, 1],$$
$$[k_1, k_2, k_3, k_4] = [2, 1, 1, 2],$$
$$[x_1(0), x_2(0), x_3(0)] = [1, 0, 0],$$
$$[x_1'(0), x_2'(0), x_3'(0)] = [0, 0, 0].$$

Three masses connected by four springs on a friction-less surface



Turbulence





This material is released by NVIDIA Corporation under the Creative Commons Attribution 4.0 International (CC BY 4.0)

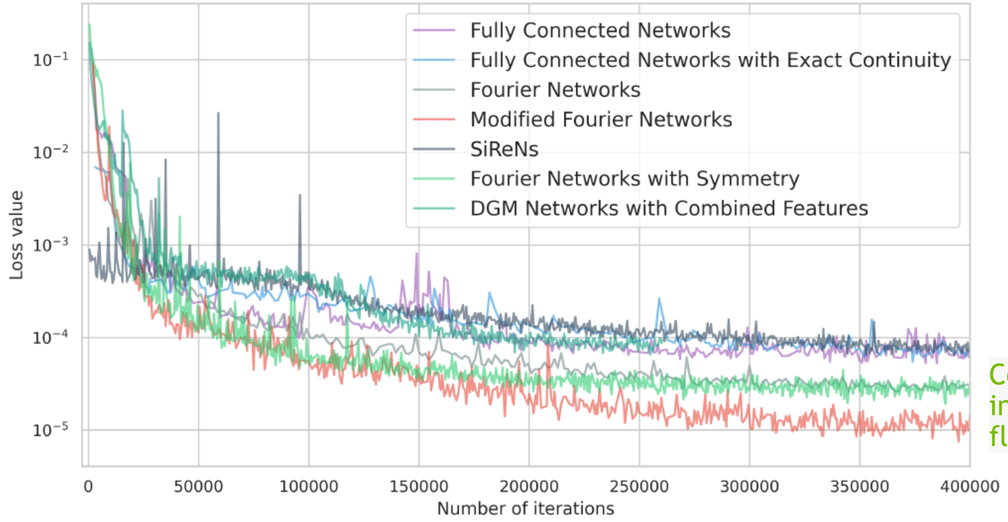
MODULUS FEATURES AND ADVANCEMENTS

Several neural network architectures:

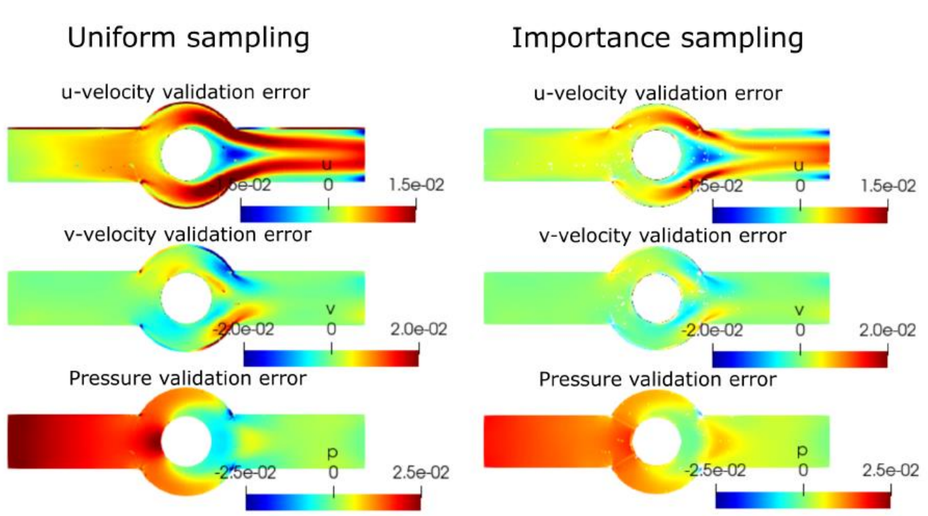
- Fully connected Network
- Fourier Feature Network
- Sinusoidal Representation Network (SiReN)
- Modified Fourier Network
- Deep Galerkin Method Network
- Modified Highway Network
- Multiplicative Filter Networks

Other Features include:

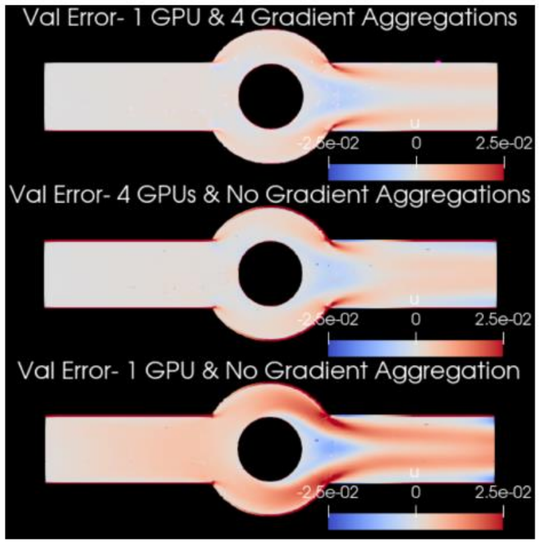
- Global and local learning rate annealing
- Global adaptive activation functions
- Halton sequences for low-discrepancy point cloud creation
- Gradient Accumulation
- Time-stepping schemes for transient problems
- Temporal loss weighting and time marching for the continuous time approach
- Importance sampling
- Homoscedastic task uncertainty quantification for loss weighting



Comparisons of various networks in SimNet applied to solve the flow over a heatsink



Importance sampling

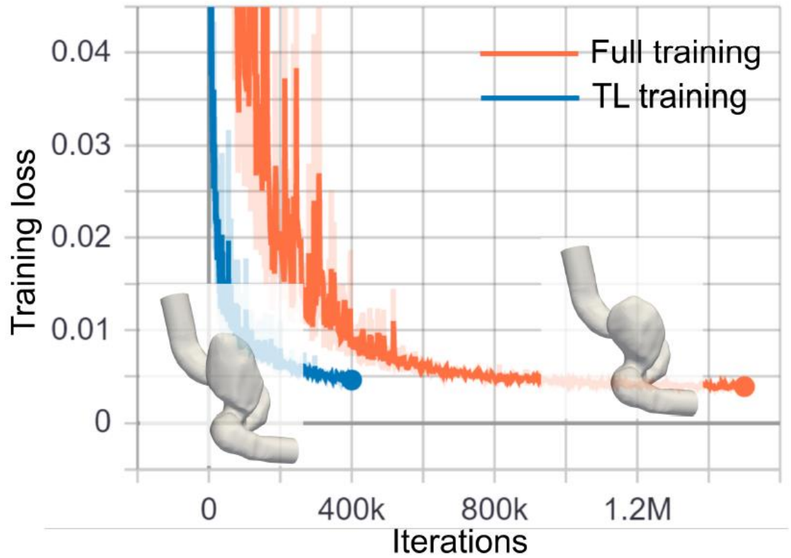


Handling larger batch sizes using Multi-GPU and/or Gradient Aggregation

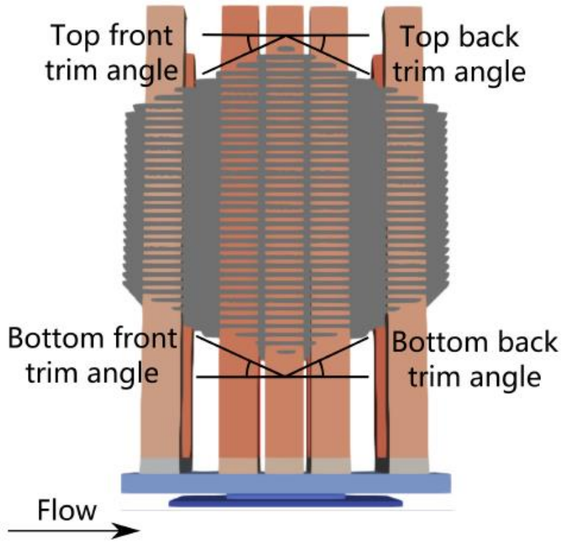
This material is released by NVIDIA Corporation under the Creative Commons Attribution 4.0 International (CC BY 4.0)

MODULUS FEATURES AND ADVANCEMENTS

- APIs to automatically generate point clouds from Boolean compositions of geometry primitives or import point cloud for complex geometry (e.g., STL files)
- Parameterized system representation that solves several configurations concurrently for analytical geometry using SimNet CSG module
- Transfer learning for efficient surrogate-based parameterization of STL and constructive solid geometries
- Polynomial Chaos Expansion method for assessing how uncertainties in a model input manifest in its output



STL geometry and Transfer Learning



Geometry parameterization using SimNet's CSG module

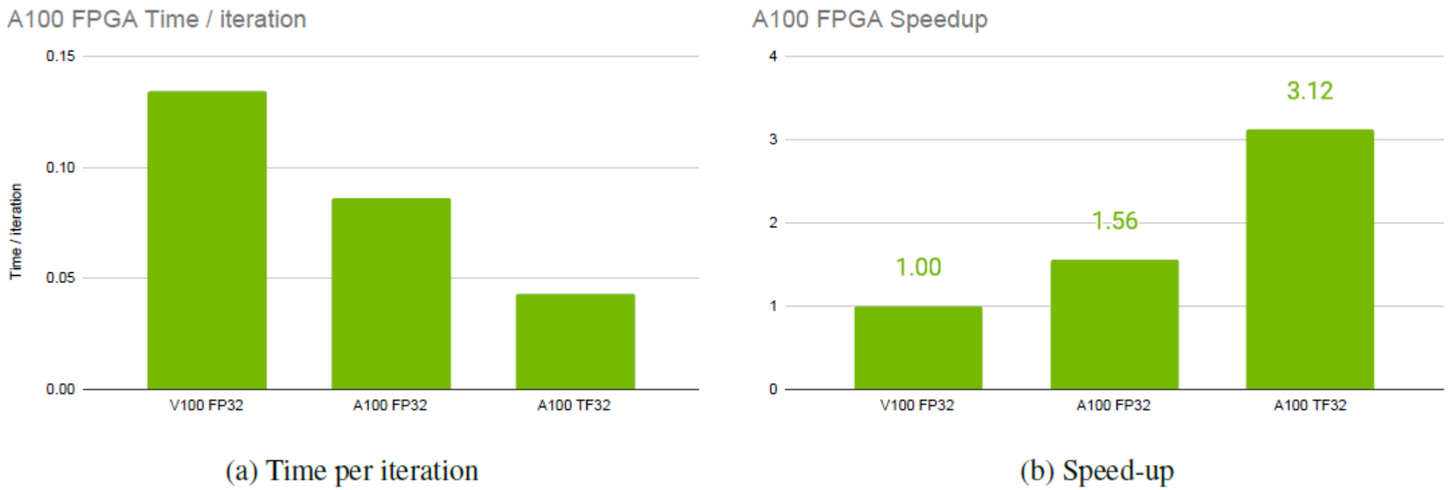
This material is released by NVIDIA Corporation under the Creative Commons Attribution 4.0 International (CC BY 4.0)

MODULUS FEATURES AND ADVANCEMENTS

Improved performance with XLA enabled for TensorFlow models and multi-GPU/multi-Node runs

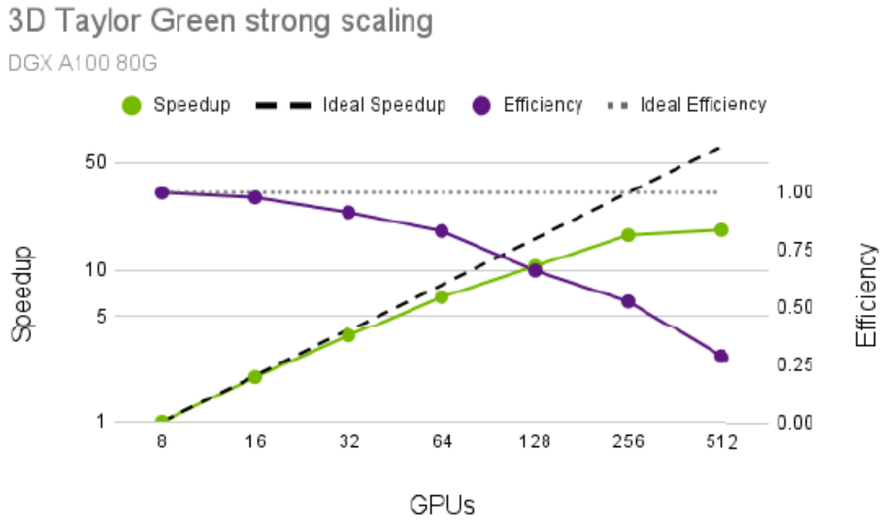
- Accelerated Linear Algebra (XLA)
- Strong scaling with learning rate adjustments

Improved stability in multi-GPU/multi-Node implementations using linear-exponential learning rate and utilization of TF32 precision for A100 GPUs



(a) Time per iteration (b) Speed-up

Accelerated training using TF32 on A100 GPUs



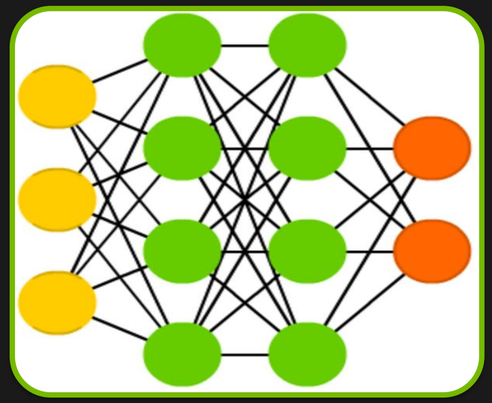
Strong Scaling- Speedup and Scaling Efficiency

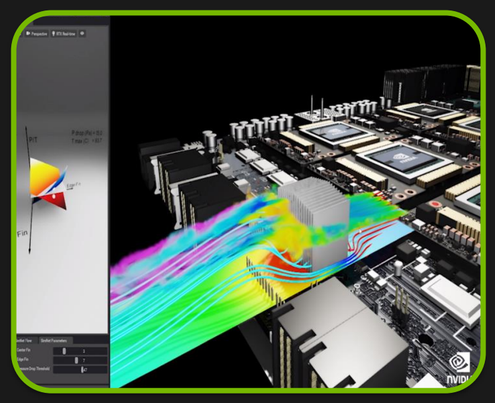
This material is released by NVIDIA Corporation under the Creative Commons Attribution 4.0 International (CC BY 4.0)

MODULUS

AI-accelerated Physics Simulation Toolkit









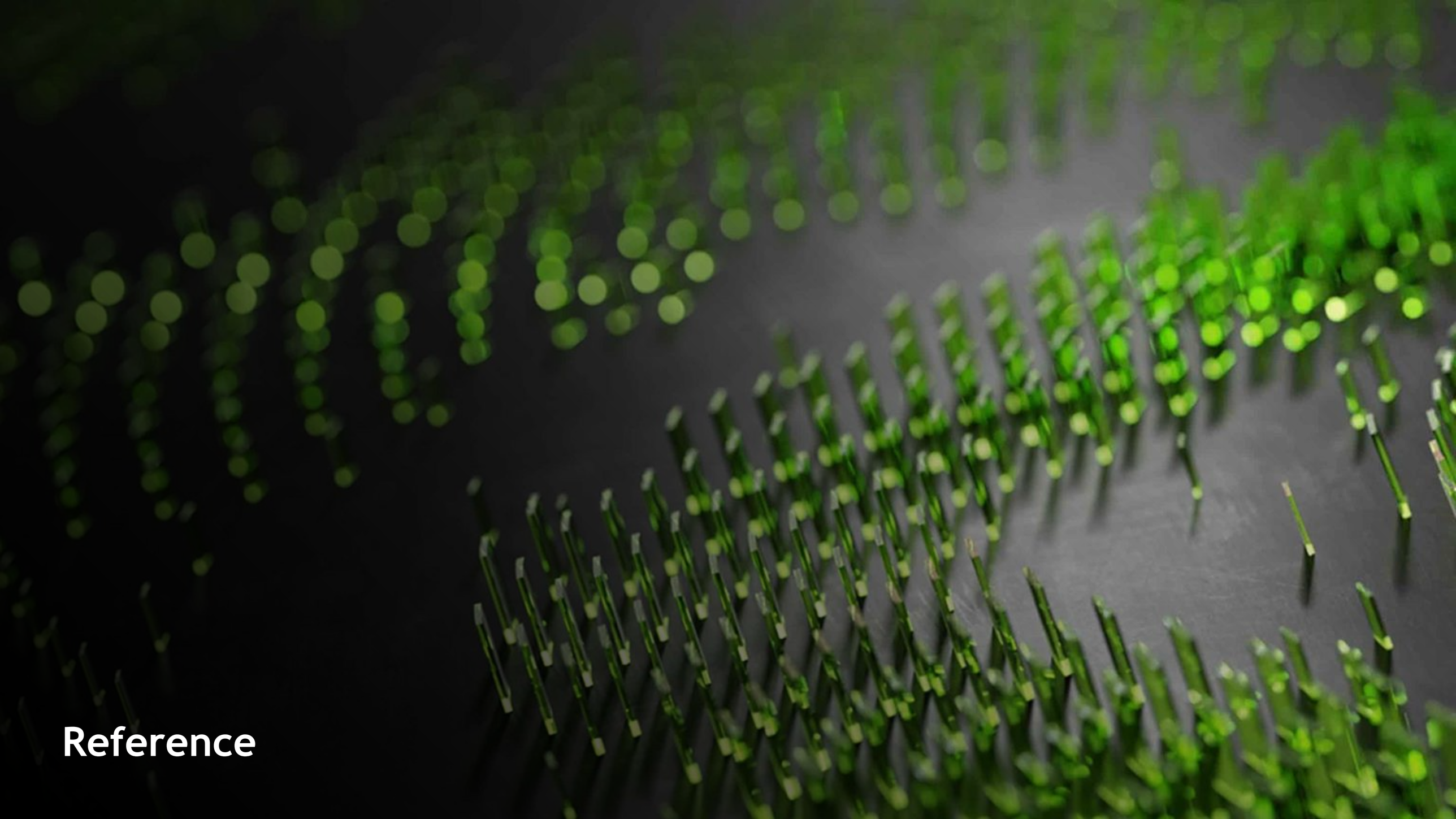
Solve larger problems faster *with* XLA, AMP and TF32 support, and Multi-GPU, Multi-Node implementation

Several advanced networks model Multiple Physics in ***Forward, Inverse and Data Assimilation simulations*** with accuracy & convergence

Parameterized system representation to solve ***multiple scenarios simultaneously***

APIs for implementing new Physics, Geometry, and Domains and detailed ***User Guide examples***

NVIDIA



Reference

SIMULATION INTELLIGENCE: TOWARDS A NEW GENERATION OF SCIENTIFIC METHODS

https://arxiv.org/abs/2112.03235

(2021/12/06)

SIMULATION INTELLIGENCE: TOWARDS A NEW GENERATION OF SCIENTIFIC METHODS

Alexander Lavin*
Institute for Simulation Intelligence

Hector Zenil
Alan Turing Institute

Brooks Paige
Alan Turing Institute

David Krakauer
Santa Fe Institute

Justin Gottschlich
Intel Labs

Tim Mattson
Intel

Anima Anandkumar
Nvidia

Sanjay Choudry
Nvidia

Kamil Rocki
Neuralink

Atılım Güneş Baydin
University of Oxford

Carina Prunkl
University of Oxford

Olexandr Isayev
Carnegie Mellon University

Erik Peterson
Carnegie Mellon University

Peter L. McMahon
Cornell University

Jakob H. Macke
University of Tübingen

Kyle Cranmer
New York University

Jiaxin Zhang
Oak Ridge National Lab

Haruko Wainwright
Lawrence Berkeley National Lab

Adi Hanuka
SLAC National Accelerator Lab

Samuel Assefa
US Bank AI Innovation

Stephan Zheng
Salesforce Research

Manuela Veloso
JPM AI Research

Avi Pfeffer
Charles River Analytics

ABSTRACT

The original "Seven Motifs" set forth a roadmap of essential methods for the field of scientific computing, where a motif is an algorithmic method that captures a pattern of computation and data movement.[1] We present the *Nine Motifs of Simulation Intelligence*, a roadmap for the development and integration of the essential algorithms necessary for a merger of scientific computing, scientific simulation, and artificial intelligence. We call this merger simulation intelligence (SI), for short. We argue the motifs of simulation intelligence are interconnected and interdependent, much like the components within the layers of an operating system. Using this metaphor, we explore the nature of each layer of the simulation intelligence "operating system" stack (SI-stack) and the motifs therein:

1. Multi-physics and multi-scale modeling
2. Surrogate modeling and emulation
3. Simulation-based inference
4. Causal modeling and inference
5. Agent-based modeling
6. Probabilistic programming
7. Differentiable programming
8. Open-ended optimization
9. Machine programming

We believe coordinated efforts between motifs offers immense opportunity to accelerate scientific discovery, from solving inverse problems in synthetic biology and climate science, to directing nuclear energy experiments and predicting emergent behavior in socioeconomic settings. We elaborate on each layer of the SI-stack, detailing the state-of-art methods, presenting examples to highlight challenges and opportunities, and advocating for specific ways to advance the motifs and the synergies from their combinations. Advancing and integrating these technologies can enable a robust and efficient hypothesis–simulation–analysis type of scientific method, which we introduce with several use-cases for human-machine teaming and automated science.

NVIDIA

REFERENCE

- View a variety of new NVIDIA GTC sessions available on Modulus (previously known as SimNet) and other similar topics:
  - SimNet: A Neural Engine for Science and Engineering Problems
  - Toward Developing High Reynolds Number, Compressible Reacting Flows in SimNet
  - Accelerate HPC Simulations at Scale with Physics-Informed Neural Networks and NVIDIA GPUs on AWS
  - Physics-Informed Neural Networks for Wave Propagation Using SimNet
  - Uncertainty Quantification for Transport in Porous Media using Parameterized Physics Informed Neural Networks
  - Energy Transition, Sustainability, and the Role of Technology (Panel discussion)
- NVIDIA SimNet Tutorial - Simulate Blood Flow in Intracranial Aneurysm (Chinese)


反饋問卷
https://forms.gle/i9pdsodoYCRuZEjL7
NVIDIA